Database Management Systems

Table of contents

1 Database Management Systems

1.0.1 Welcome to DBMS (CS2001) Course

This portal contains all the materials, interactive playgrounds, and textbook references for the Database Management Systems course.

1.0.1.1 Course Overview

- **Course ID:** BSCS2001
- **Course Type:** Programming
- **Course Credits:** 4 Credits
- **Pre-requisites:** None

1.0.1.2 Textbook

- **Title:** *Database System Concepts* (7th Ed.)
- **Authors:** Silberschatz, Korth, Sudarshan
- **Publisher:** McGraw-Hill

1.0.2 Course Faculty

Dr. Partha Pratim Das
Professor, Department of Computer Science and Engineering, IIT Kharagpur

1.0.3 Course Instructors:

Manidipa

Pravin

Bhaskar

Arshpreet

1.0.4 How to Use This Portal

The content is organized chronologically. Just expand the navigation menu on the left sidebar or click **Start Course** above to jump into specific topics.

Part I

Week-01

2 Introduction

2.1 Key Terminologies

- **Data:** This is just raw, unorganized information without any extra context.
- **Record:** A collection of related data fields packaged together to describe one single real-world object. For example, grouping an instructor's ID, their name, and their department into a single cohesive row.
- **File:** A collection of related records.
- **Database:** The collection of structured information relevant to an enterprise is referred to as the **database**.

A **Database Management System (DBMS)** is a collection of interrelated data along with a set of programs designed to store, manage and access the data efficiently and conveniently.

> A DBMS is typically used to manage data that is highly valuable, relatively large and accessed by multiple users at the same time. **Applications** : Sales, Manufacturing, Banking, Airlines, University etc.

2.2 Why DBMS?

Historically, companies kept their permanent records in separate flat text files managed by conventional operating systems. Doing this without a centralized DBMS creates massive problems:

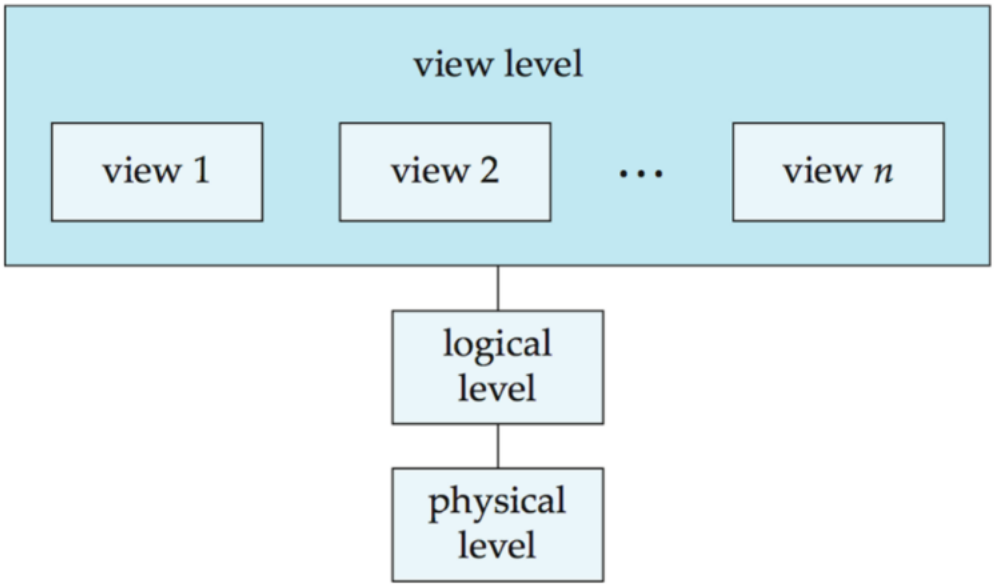
- **Data Redundancy & Inconsistency:** Separate departments build their own unique files over time. If a student changes their address, it gets updated in one file but remains outdated in another, leading to conflicting records.
- **Difficulty in Accessing Data:** Standard files aren't flexible. If a manager asks for a quick, unique list that the original developer didn't explicitly anticipate, a programmer has to write a brand-new script from scratch. It's slow and unresponsive.

- **Data Isolation:** Data gets scattered across disconnected files with conflicting file layouts and disparate formats, making it incredibly complex to build new applications that aggregate data.
- **Integrity Problems:** Real world rules, like a bank balance never dropping below zero, must be manually hardcoded into every single application script. If the rule changes, editing hundreds of decoupled programs is prone to human error.
- **Atomicity Issues:** Computer hardware is vulnerable to power failures or crashes. Database operations must be "all-or-nothing" (atomic). If a system crashes halfway through a fund transfer, a file system makes it very difficult to cleanly roll back the changes.
- **Concurrent-Access Anomalies:** To keep response times fast, systems must let multiple processes update data simultaneously. Without a centralized coordination engine, separate programs independently reading and writing raw text files will crash into each other and overwrite each other's updates.
- **Security Flaws:** Enforcing granular security (like letting a user see specific fields or rows but hiding others) is almost impossible because the underlying operating system doesn't understand the internal semantics of the data files.

3 View of Data

3.0.1 Data Abstraction

Abstraction refers to the hiding of implementation details to simplify the user interactions with the system.



- **Physical level** : Describes *how* the data is actually stored. It describes complex low-level data structures in detail.
- **Logical level** : Describes *what* data is stored in the database, along with the relationships that exist between them.
- **View level** : This is the highest level of abstraction that simplifies the user interaction by describing only the necessary part of the database. The system may provide different views for the same database.

3.0.2 Schema and Instance

- ***Schema*** refers to the overall design of the database. Database systems have several schemas, partitioned according to the levels of abstraction:
  - **Physical schema** : Describes the overall physical design of the database at the physical level.
  - **Logical schema** : Describes the overall logical design of the database at the logical level.

Name	Customer ID	Account #	Aadhaar ID	Mobile #

- ***Instance*** refers to the collection of information stored in the database at a given point of time.

Name	Customer ID	Account #	Aadhaar ID	Mobile #
Pavan Laha	6728	917322	182719289372	9830100291
Lata Kala	8912	827183	918291204829	7189203928
Nand Prabhu	6617	372912	127837291021	8892021892

- ***Physical Data Independence*** refers to the ability to modify the physical schema without changing the logical schema. This ensures that the different interfaces are defined such that any changes in one of the levels does not seriously influence others.
- ***Logical Data Independence*** refers to the ability to modify the logical schema without affecting the view level.

3.1 Database Languages

- **Data Definition Language (DDL)** :
  - The Data-Definition Language is used to specify the database schema and add structural properties or constraints to the data.
  - The processing of DDL statements generates metadata (data about data), which is stored securely in a centralized data dictionary. This directory is a special set of system tables that regular users cannot modify directly; the database system consults it before reading or altering any live data.
  - Example:

```
create table department(dept_name char(20),
                        building char(15),
                        budget numeric(12,2),
                        primary key (dept_name));
```

- **Data Manipulation Language (DML)** :
  - The Data-Manipulation Language enables users to access or alter data organized under the schema. DML handles information retrieval (queries), insertions, deletions, and updates.
  - Example:

```
select instructor.name
from instructor
where instructor.dept_name = 'History';
```

4 Database Design

Database design primarily involves designing the database schema to manage large, interconnected bodies of enterprise information.

- **User Requirements Specification** : Fully characterize and document the data needs of prospective database users.
- **Conceptual Design** : Translate user requirements into a high-level, detailed conceptual schema of the database, focusing entirely on data modeling rather than physical implementation constraints.
- **Logical Design** : Map the abstract, high-level conceptual schema onto the system-specific implementation data model of the chosen Database Management System (DBMS).
- **Physical Design** : Specify the internal and low-level physical features of the database configuration.

4.1 Database Engine

A database system is partitioned into independent modules that isolate core responsibilities. The database engine can be broadly divided into three main functional segments: the storage manager, the query processor, and the transaction manager.

4.1.1 Storage Manager

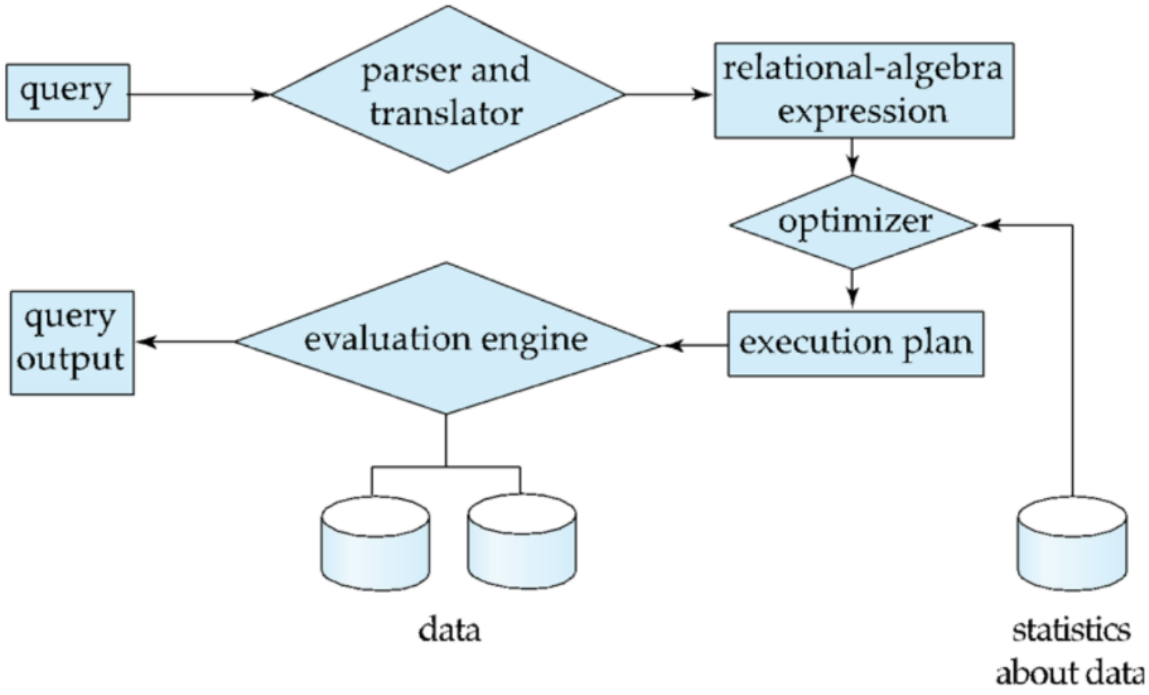
The storage manager acts as the operating interface between the low-level data stored on disk hardware and the application programs or queries submitted to the system.

- Validates user access permissions and tests whether newly modified data satisfies established consistency constraints.
- Keeps the global database in a correct state despite hardware crashes and prevents operational conflicts between concurrent tasks.
- Directs the physical allocation of disk space and manages the underlying byte structures used to represent database records.

- Coordinates caching strategies, determining when to fetch data from physical storage into volatile main memory so data pools can scale larger than the computer's actual RAM capacity.

4.1.2 Query Processor

The query processor abstracts execution details so developers can focus on writing high-level declarative commands, translating text into optimized sequences of lower-level physical operations.



- **Parsing and Translation** : When you submit a query, the system can't read it as raw text. The DDL Interpreter handles structural commands and updates the data dictionary. For data modifications, the DML Compiler parses your syntax for correctness and translates the declarative text into a relational-algebra expression (an internal mathematical representation the system can work with).
- **Optimization** : A single query can be executed using many alternative strategies, called evaluation plans. They all give the exact same output, but some consume drastically less hardware resources. The Query Optimizer calculates the estimated computational cost (disk I/O and CPU time) for each plan and programmatically selects the absolute most efficient one.

- **Evaluation** : The execution layer takes over. The Query Evaluation Engine takes those optimized, low-level instructions, executes them directly against the physical database files, and returns a clean result table back to the user.

4.1.3 Transaction Management

A transaction is a collection of separate program operations that perform a single logical function within an application. The transaction manager allows developers to treat multiple database updates as a single unit of work by enforcing the core ACID properties.

- Maintains system safety and manages failure recovery. It automatically detects system crashes and uses logging protocols to reverse or restore data back to the clean state that existed prior to the failed transaction.

Part II

Week-02

5 Relational Model & DDL

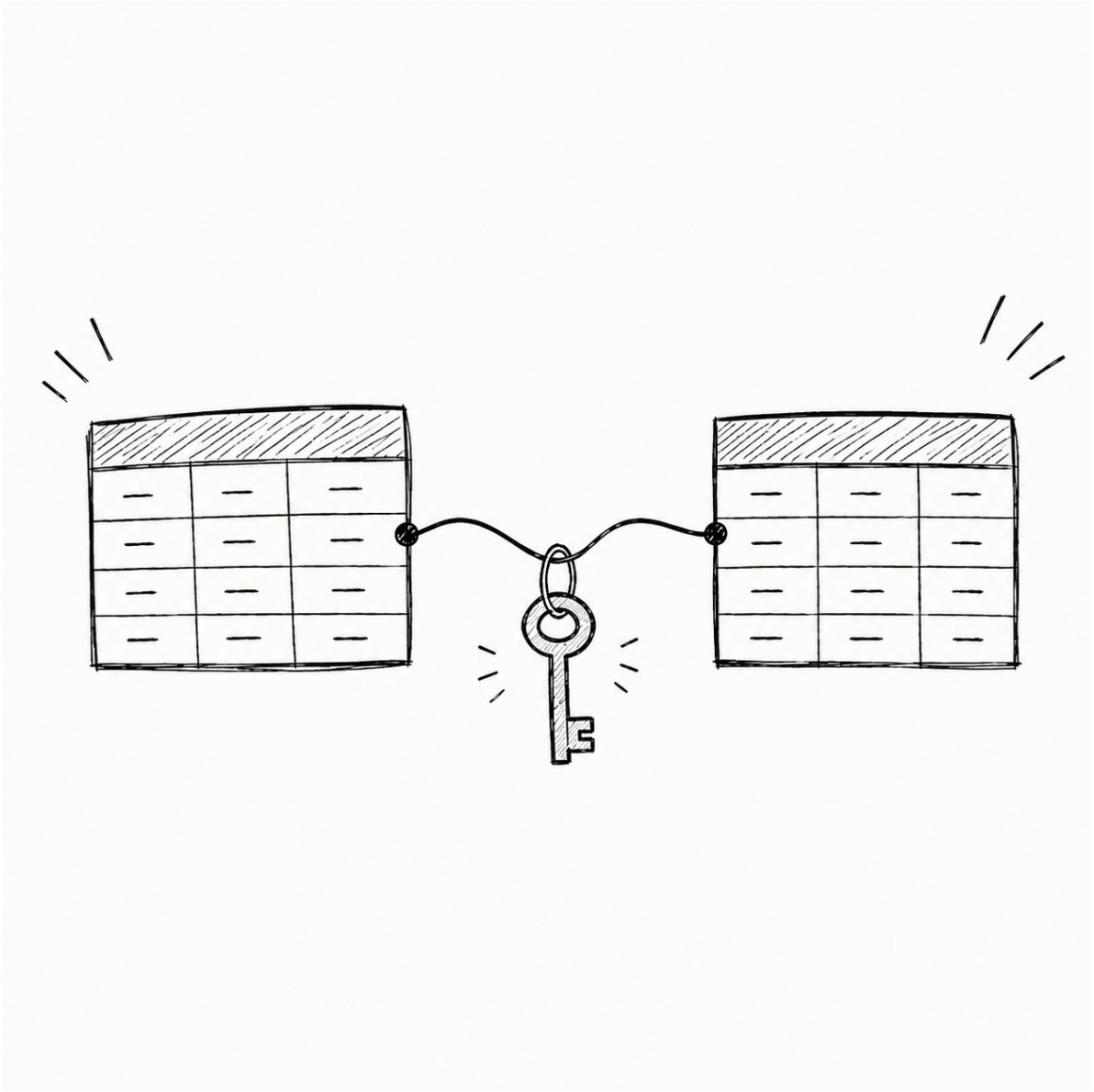


Figure 5.1: Relational Database Concepts

5.1 What is a Relational Database?

Before diving into commands, let's build some intuition. Imagine a school register. You have a list of students, their roll numbers, and the courses they are taking. In a **Relational Database**, we store data in structured **tables** (also called *relations*). Every row represents a single entity (like one specific student), and every column represents an attribute of that entity (like their name or grade).

The true power of relational databases isn't just storing tables—it's how we can *relate* them using Keys, and slice them using mathematical operations!

5.2 The Concept of Keys

In a database, we need a way to uniquely identify every row so we don't confuse two students named "John Smith". We do this using **Keys**.

- **Super Key**: Any combination of columns that can uniquely identify a row. For a student, `[Roll Number, Name]` is a super key, but it's bulkier than it needs to be.
- **Candidate Key**: A *minimal* Super Key. It has no unnecessary columns. Both `[Roll Number]` and `[Email Address]` could be candidate keys.
- **Primary Key**: The one Candidate Key you officially choose to identify your rows. Once chosen, no two rows can have the same Primary Key, and it cannot be empty.
- **Surrogate Key**: Sometimes, natural data (like names) makes for bad keys. A Surrogate Key is an artificially generated ID (like an auto-incrementing `Student_ID = 1, 2, 3...`) created purely to act as the Primary Key.
- **Foreign Key**: This is the literal glue that makes a database "relational"! A Foreign Key is a column in Table A that points exactly to the Primary Key in Table B. It links the two tables together.

Example: The `student_id` column in the `Grades` table is a Foreign Key that points to the `student_id` Primary Key in the `Student` table.

5.3 Relational Queries & The Pure Language

How do we ask a database for information? We use a **Query Language**.

There's a fundamental difference in how these languages work:

- **Imperative Languages** (like Python or Java): You must write step-by-step instructions on *how* to find the data.
- **Declarative Languages**: You simply describe *what* data you want, and the system figures out the best way to get it.

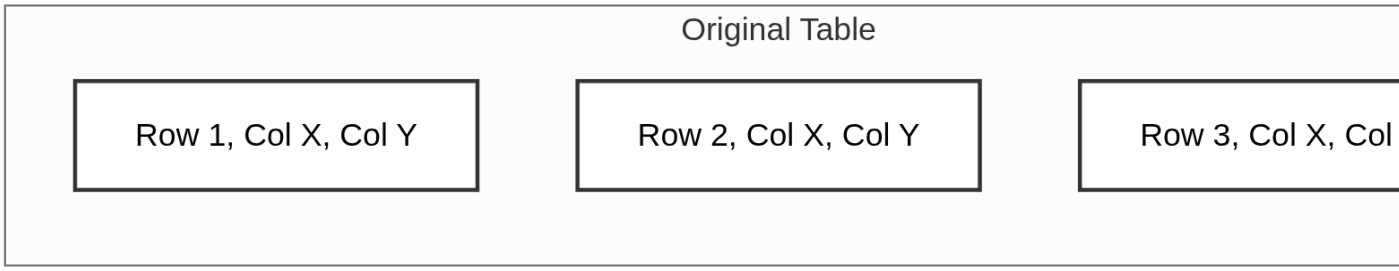
Before modern SQL existed, computer scientists created "pure" theoretical languages to prove databases could work. **Relational Algebra** is one of these pure languages. It uses mathematical symbols to describe exactly how to slice and dice tables.

5.4 Relational Operators: Slicing and Dicing

Relational Algebra gives us **Relational Operators**—surgical tools to extract what we need.

5.4.1 The Big Two: Select vs. Project

- **Select (σ)**: Filters **Rows**. It scans horizontally. (e.g., "Find all students who got an 'A' ").
- **Project (π)**: Filters **Columns**. It scans vertically. (e.g., "I only need to see Names and Roll Numbers").



5.4.2 Other Important Operators

Operator	Symbol	Intuition	Everyday Example
Union	$\cup$	Combines two sets into one big list.	"Give me the list of students in the morning class PLUS the evening class."
Intersection	$\cap$	Returns items present in *both* sets.	"Who is taking BOTH Physics AND Chemistry?"
Set Difference	$-$	Returns items in the first set but NOT in the second.	"Students taking Physics but NOT taking Chemistry."
Cartesian Product	$\times$	Combines *every* row of one table with *every* row of another.	Matching every color of a shirt with every size available.

Operator	Symbol	Intuition	Everyday Example
Natural Join	$\bowtie$	Connects two tables seamlessly based on matching columns.	Linking the `Student` table to the `Grades` table.

5.5 What is SQL?

While Relational Algebra is great for theory, we needed a real, practical language to type into a computer. Enter **SQL** (Structured Query Language).

A Brief History: Developed at IBM in the 1970s by Donald Chamberlin and Raymond Boyce, SQL was initially called "SEQUEL". It was designed to manipulate data stored in IBM's quasi-relational database management system. Over the decades, it became the undisputed, standardized language for relational databases globally.

SQL is broadly divided into three main categories:

1. **DDL (Data Definition Language)**: Used to define the structure (the blueprint).
2. **DML (Data Manipulation Language)**: Used to insert, update, and query the actual data (the furniture).
3. **DCL (Data Control Language)**: Used to manage permissions and security (the bouncers).

5.6 Domain Types & Integrity Constraints

Before building our table, we must set the rules.

5.6.1 Domain Types

We must tell the system *what kind* of data a column stores. This prevents someone from typing "Twenty" into an Age column.

Type	Description	Example
`char(n)`	Fixed-length text. Good for Zip Codes.	`char(5)` pads "Hi" to `"Hi "`
`varchar(n)`	Variable-length text. Good for names.	`varchar(50)` stores "Hi" as `"Hi"`
`int`	A whole number.	`25`
`numeric(p,d)`	A decimal number. `p` is total digits, `d` is after the decimal.	`numeric(4,2)` allows `44.22` but rejects `100.22`

5.6.2 Integrity Constraints

Constraints are extra rules we apply to columns to ensure the data makes sense.

- `NOT NULL`: The column cannot be left empty.
- `UNIQUE`: No two rows can have the same value in this column.
- `CHECK(condition)`: Ensures the value meets a specific requirement (e.g., `CHECK (age >= 18)`).
- `PRIMARY KEY`: Automatically enforces both `UNIQUE` and `NOT NULL`.

5.7 DDL: Building the Blueprint

Intuition: DDL is the architect. You use DDL to draw the blueprints and set the rules. DDL does **not** handle the actual data.

5.7.1 1. The `CREATE` Command

We use `CREATE TABLE` to bring a new table into existence. Let's create a `student` table with constraints!

```
CREATE TABLE student (
    roll_no varchar(10),
    name varchar(50) NOT NULL,
    age int CHECK (age > 10),
    email varchar(100) UNIQUE,
    primary key (roll_no)
);
```

What happened here?

- `name` cannot be left blank (`NOT NULL`).
- `age` must be greater than 10. If someone tries to insert a 9-year-old, the database will block it!
- `email` must be `UNIQUE` across all students.
- `roll_no` is our `PRIMARY KEY`.

5.7.2 2. The `ALTER` Command

What if we built our table, but later realized we forgot to add a phone number column? We use `ALTER` to carefully modify the blueprint without destroying the table.

```
-- Adding a new column to an existing table
ALTER TABLE student ADD phone_number varchar(15);

-- Changing the data type of an existing column
ALTER TABLE student ALTER COLUMN name varchar(100);
```

5.7.3 3. The `DROP` Command

When a table is completely obsolete, we use `DROP`.

```
DROP TABLE student;
```

> **Warning:** Use `DROP` very carefully! It wipes out the table structure and everything inside it instantly.

6 Data Manipulation (DML) & Basic Queries

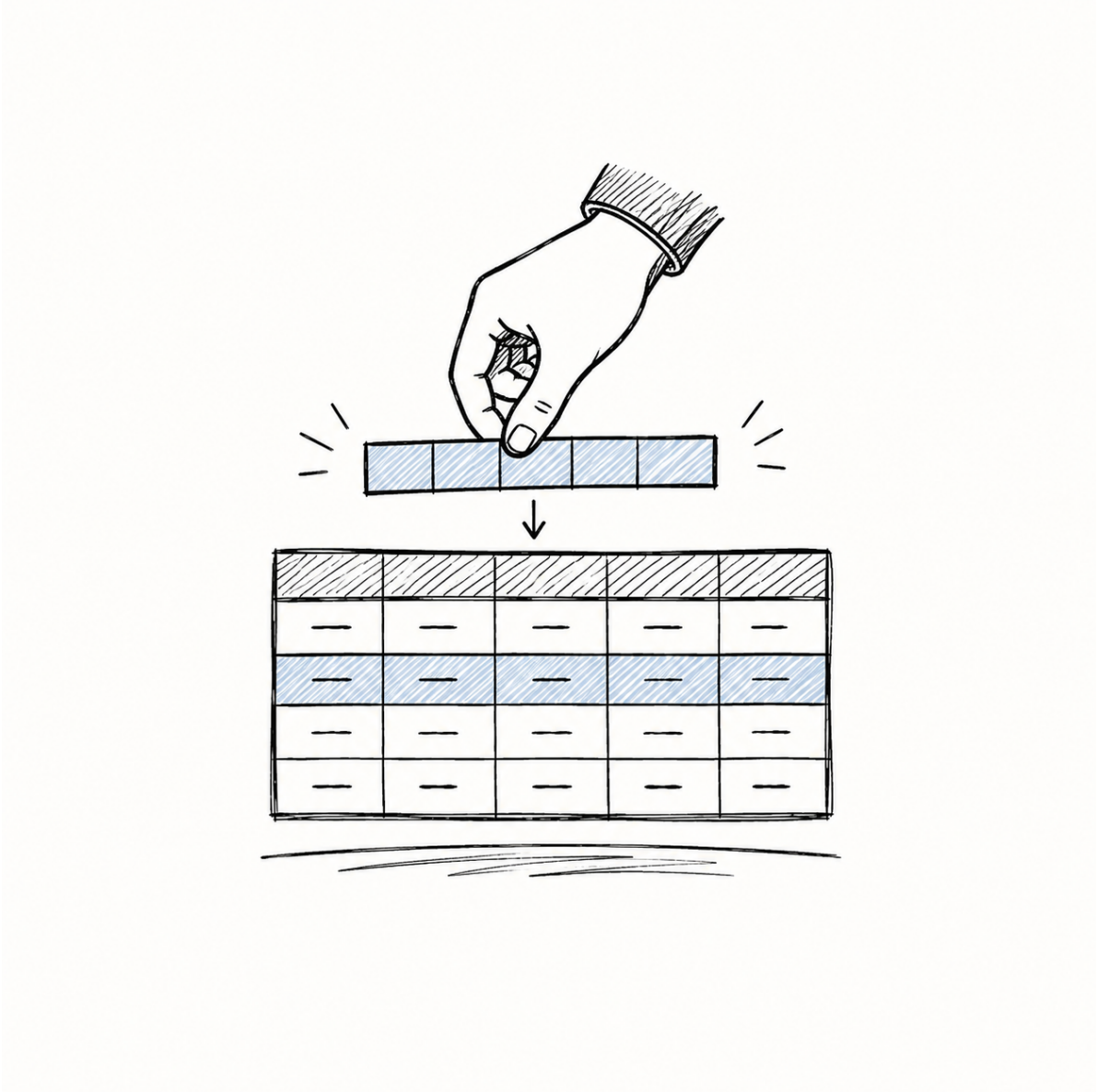


Figure 6.1: DML Operations

6.1 DML: Furnishing the House

In the previous section, DDL acted as our architect—building the empty structure of our `student` table. Now, it's time for **DML (Data Manipulation Language)** to move the furniture in! DML handles all the inserting, modifying, and deleting of the *actual data rows*.

Let's start with an empty table:

roll_no	name	level	course	term	grade
(empty)					

6.1.1 1. `INSERT`: Adding New Records

A new student, Pravin, has just enrolled. We need to add him to our database.

```
INSERT INTO student (roll_no, name, level, course, term, grade)
VALUES ('26f10', 'Pravin', 'Degree', 'DBMS', '27t1', 'S');
```

> **Tip:** You *can* write an `INSERT` without listing the column names (e.g., `INSERT INTO student VALUES (...)`), but specifying the columns as shown above is much safer! It prevents errors if someone later adds a new column to the table.

Inserting Multiple Rows at Once: You can also insert several students in a single command by separating the values with commas!

```
INSERT INTO student (roll_no, name, level, course, term, grade)
VALUES
    ('26f11', 'Alice', 'Degree', 'Math', '27t1', 'B'),
    ('26f12', 'Bob', 'Diploma', 'Physics', '27t1', 'C');
```

Our Table Now:

roll_no	name	level	course	term	grade
26f10	Pravin	Degree	DBMS	27t1	S

6.1.2 2. `UPDATE`: Modifying Existing Data

Oops! We made a mistake. Pravin actually got an 'A' grade, not an 'S'. We don't delete his record; we just `UPDATE` it.

```
UPDATE student
SET grade = 'A'
WHERE roll_no = '26f10';
```

> [**WARNING**] Always, *always* use a `WHERE` clause with `UPDATE`!

What happens if you forget the `WHERE` clause? If you simply ran `UPDATE student SET grade = 'A';`, it would ignore the individual student and change the grade of **every single student** in the entire database to an 'A'!

roll_no	name	grade
26f10	Pravin	**A**
26f11	Alice	**A**
26f12	Bob	**A**

(Oops! Alice and Bob just got free 'A' grades because you forgot the WHERE clause!)

Our Table After Correct Update:

roll_no	name	level	course	term	grade
26f10	Pravin	Degree	DBMS	27t1	**A**

6.1.3 3. `DELETE`: Removing Records

Pravin has decided to transfer to another school. We need to remove his record completely.

```
DELETE FROM student
WHERE roll_no = '26f10';
```

(Just like `UPDATE`, forgetting the `WHERE` clause here would delete everyone!)

6.2 The Anatomy of a Query

Now that we have data, how do we ask the database questions? We write queries.

Think of a SQL query like giving instructions to an incredibly fast, but very literal librarian. The fundamental structure requires three clauses:

```
SELECT <attributes>
FROM <tables>
WHERE <condition>;
```

Here is the mental model of how the librarian thinks: 1. **`FROM`**: *"Which section of the library (tables) should I go to?"* 2. **`WHERE`**: *"Which specific books (rows) should I pull off the shelf?"* 3. **`SELECT`**: *"Which chapters (columns) should I photocopy and give to you?"*

6.2.1 The `SELECT` Clause in Detail

The `SELECT` clause dictates what the final output looks like.

- `*` : "Give me everything." (All columns)
- `DISTINCT` : "Remove any duplicate answers."
- `AS` : "Rename this column in the output just to make it look nicer."
- `TOP <n>` (or `LIMIT`): "Only show me the first `n` results."

Example:

```
SELECT DISTINCT(name) AS Instructor_Name
FROM instructor
WHERE dept_name = 'Biology';
```

6.2.2 The `ORDER BY` Clause

By default, SQL returns rows in whatever internal order it feels like. If you want them sorted, you must explicitly use the `ORDER BY` clause at the very end of your query.

- `ASC`: Ascending order (A-Z, 1-10). This is the default.
- `DESC`: Descending order (Z-A, 10-1).

Example:

```sql
-- Sort instructors by salary from highest to lowest
SELECT name, salary
FROM instructor
ORDER BY salary DESC;
```

6.3 The Danger of Cross Joins

What happens if you tell the librarian to get data from *two* places at once?

```sql
SELECT *
FROM instructor, course;
```

When you list multiple tables in the `FROM` clause without a `WHERE` condition, SQL performs a **Cross Join** (Cartesian Product). It pairs *every* row in the first table with *every* row in the second table.

Let's visualize this: If you have 2 Instructors and 2 Courses:

Instructors Table:

id	name
1	Alice
2	Bob

Courses Table:

c_id	title
101	Math
102	Science

The Cross Join Result:

id	name	c_id	title
1	Alice	101	Math
1	Alice	102	Science
2	Bob	101	Math

id	name	c_id	title
2	Bob	102	Science

(2 instructors × 2 courses = 4 rows!)

Imagine if you had 1,000 instructors and 1,000 courses—you'd instantly generate **1 million rows**! Most of these combinations are nonsense (Alice doesn't teach every single course).

6.3.1 Fixing the Cross Join

To fix this, we use the `WHERE` clause to filter the massive Cross Join down to only the combinations that make sense (e.g., linking the instructor to the specific course they actually teach):

```
SELECT *
FROM instructor AS i, course AS c
WHERE i.course_id = c.course_id
  AND i.dept_name = 'Biology';
```

(Notice how we used `AS i` and `AS c` to give our tables nicknames? This makes our `WHERE` clause much easier to read!)

7 Advanced SQL Filtering, Sets & Aggregation

Figure 7.1: Advanced SQL Filtering

7.1 Advanced Filtering

Sometimes, a simple `=` or `>` isn't enough. What if you only remember that an instructor's name *starts with* an 'M', or you need to check if their salary falls within a specific range?

7.1.1 1. String Operations (`LIKE`)

`LIKE` is SQL's built-in detective. It performs pattern matching on text using two special wildcard characters:

- `%` : Matches **any sequence** of characters (even zero characters).
- `_` : Matches exactly **one** character.

Examples in Action:

- `LIKE 'Intro%'` : Matches any string starting with "Intro" (e.g., "Introduction", "Intro to CS").
- `LIKE '%Comp%'` : Matches any string containing "Comp" anywhere inside it (e.g., "Computer", "MyComp").
- `LIKE '___'` (three underscores) : Matches any string of exactly three characters (e.g., "Cat", "Dog").

```
SELECT name
FROM instructor
WHERE name LIKE '%dar%';
-- Finds names like 'Mandar', 'Darren', 'Cedar'
```

7.1.2 2. Advanced Predicates (`BETWEEN` and `IN`)

These are essentially shortcuts (syntax sugar) that save you from writing long, messy `WHERE` clauses.

`BETWEEN a AND b`: A clean way to check for a range (inclusive of both `a` and `b`).

```
-- Instead of: WHERE salary >= 90000 AND salary <= 100000
SELECT name
FROM instructor
WHERE salary BETWEEN 90000 AND 100000;
```

`IN`: A shorthand for multiple `OR` conditions.

```sql
-- Instead of: WHERE dept_name = 'Comp Sci' OR dept_name = 'Biology'
SELECT name
FROM instructor
WHERE dept_name IN ('Comp Sci', 'Biology');
```

7.2 Dealing with the Unknown: `NULL`

In databases, what happens when we simply *don't know* a value? For example, a new student hasn't been assigned a dorm yet. We don't enter "0" or "None" because those are actual values. Instead, we use `NULL`.

`NULL` signifies a missing or unknown value. It is **not** empty text, and it is **not** the number zero.

7.2.1 The `IS` Operator

Because `NULL` means "unknown", we cannot use standard mathematical operators like `=` or `!=` on it.

If you ask SQL: `WHERE dorm = NULL`, the result is practically "Unknown = Unknown?", which doesn't make sense! Instead, we must use the special `IS` operator.

```sql
-- Find students who haven't been assigned a dorm yet
SELECT name
FROM student
WHERE dorm IS NULL;

-- Find students who HAVE a dorm
SELECT name
FROM student
WHERE dorm IS NOT NULL;
```

7.2.2 Three-Valued Logic

Because of `NULL`, SQL doesn't just use standard boolean logic (True / False). It uses **Three-Valued Logic**:

1. **True**

2. **False**
3. **Unknown** (The result of comparing something to `NULL`)

Here's how `Unknown` impacts your queries:

- `True AND Unknown` → `Unknown`
- `False AND Unknown` → `False` (Because False ruins everything in an AND)
- `True OR Unknown` → `True` (Because True saves everything in an OR)
- `False OR Unknown` → `Unknown`

  Critical Rule: The `WHERE` clause will *only* return rows where the final condition evaluates to exactly **True**. If the condition evaluates to **False** or **Unknown**, the row is thrown out!

7.3 Set Operations

Because relational databases are built on mathematical Set Theory, we can perform operations on the results of two different queries!

Imagine Query A returns `[Alice, Bob, Charlie]` and Query B returns `[Bob, Charlie, David]`.

Operator	Action	Result
UNION	Combines both sets, removing duplicates.	`[Alice, Bob, Charlie, David]`
INTERSECT	Keeps only what is present in *both* sets.	`[Bob, Charlie]`
EXCEPT	Keeps what is in Set A, but *subtracts* anything found in Set B.	`[Alice]`

Note: By default, these operations remove duplicate rows. If you want to keep duplicates, add the `ALL` keyword (e.g., `UNION ALL`).

7.4 Aggregation & Grouping

Often, we don't care about the individual rows. If a university has 50,000 students, we might just want to know the *total* number of students or the *average* GPA.

Aggregate functions take a whole column of data, mash it together, and return a single summary value.

- `avg()`: Average value
- `min()`: Minimum value
- `max()`: Maximum value
- `sum()`: Sum of all values
- `count()`: Count of rows

7.4.1 `GROUP BY`: Sorting into Buckets

What if we want the average salary *for every single department*? We can't just use `avg(salary)` alone, because that gives us the average of the entire university.

Instead, we use `GROUP BY`. This tells SQL: "Sort the rows into buckets based on their department. Then, calculate the average for each bucket independently!"

```
SELECT dept_name, avg(salary)
FROM instructor
GROUP BY dept_name;
```

7.4.2 `HAVING`: The Bouncer for Buckets

We already learned that `WHERE` filters individual rows. But what if we want to filter the *groups* we just created? For example, "Show me departments where the *average salary* is greater than 42,000."

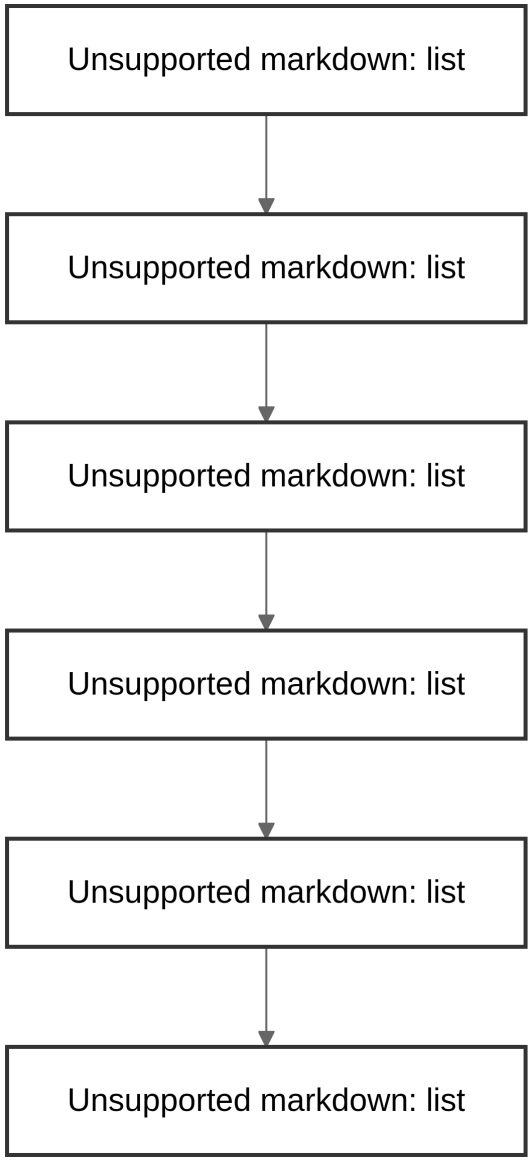
You cannot use `WHERE avg(salary) > 42000` because `WHERE` happens *before* the buckets are created!

Instead, we use `HAVING`. If `WHERE` is the bouncer for rows, `HAVING` is the bouncer for groups!

```
SELECT dept_name, avg(salary)
FROM instructor
GROUP BY dept_name
HAVING avg(salary) > 42000;
```

7.5 The SQL Execution Order

When you write a complex SQL query, the database does *not* read it from top to bottom! It processes the clauses in a very specific order. Understanding this order is the ultimate secret to mastering SQL.



Step-by-Step Narrative:

1. `FROM`: The engine goes to the tables to gather the raw data.
2. `WHERE`: It filters out the individual rows that don't meet the criteria.
3. `GROUP BY`: It takes the surviving rows and sorts them into buckets.
4. `HAVING`: It looks at the buckets, calculates the aggregates, and throws away any buckets that fail the test.
5. `SELECT`: It finally decides which columns to show you on the screen.
6. `ORDER BY`: It alphabetizes or sorts the final output for you.

Part III

Week-03

8 Nested Subqueries

8.1 Nested Subqueries

A **subquery** is simply a `select-from-where` block placed *inside* another query. SQL lets you nest subqueries in the `WHERE`, `FROM`, and `SELECT` clauses.

- `select` : Must return a single scalar value.
- `from` : Can return any valid relation (table).
- `where` : Used in expressions like `Attribute <operation> (subquery)`.

8.2 Subqueries in the `WHERE` Clause

Typically used to test **set membership**, **set comparison**, and **set cardinality**.

8.2.1 Set Membership: `IN` / `NOT IN`

Example: Find employees who worked on Project A **and** Project B.

```
select distinct emp_id
from assignment
where project = 'Project A' and
      emp_id in (select emp_id
                 from assignment
                 where project = 'Project B');
```

Swap `in` for `not in` to find employees on Project A **but not** Project B.

8.2.2 Set Comparison: `SOME` and `ALL`

Clause	Meaning	Logic
`some`	True if comparison holds for **at least one** row in subquery	$\exists\, t \in r : F\langle \text{comp} \rangle\, t$
`all`	True if comparison holds for **every** row in subquery	$\forall\, t \in r : F\langle \text{comp} \rangle\, t$

<comp> can be: <, , >, , =,

Example: Employees earning more than **some** employee in Sales:

```
select name
from employee
where salary > some (select salary
                     from employee
                     where dept_name = 'Sales');
```

Example: Employees earning more than **all** employees in Sales:

```
select name
from employee
where salary > all (select salary
                    from employee
                    where dept_name = 'Sales');
```

8.2.3 Test for Empty Sets: `EXISTS`

- `exists r` is true if `r` is non-empty.
- `not exists r` is true if `r` is empty.

Example: Find projects active in both Q1 2025 and Q2 2025, using a **correlated subquery** (inner query refers to the outer query's variable).

```
select project_id
from timeline as S
where quarter = 'Q1' and year = 2025 and
      exists (select *
              from timeline as T
              where quarter = 'Q2' and year = 2025
                    and S.project_id = T.project_id);
```

- `S` is the **correlation name** (outer query variable)
- The inner query referencing `S` is the **correlated subquery**

Example: Find employees who have worked on **all** projects in the Marketing department (uses `not exists` + set difference logic):

```
select distinct S.emp_id, S.name
from employee as S
where not exists (
    (select project_id from project where dept_name = 'Marketing')
    except
    (select T.project_id from assignment as T where S.emp_id = T.emp_id)
);
```

8.2.4 Test for Duplicates: `UNIQUE`

`unique (subquery)` is true if the subquery's result has **no duplicate rows**.

```
select T.project_id
from project as T
where unique (select R.project_id
              from assignment as R
              where T.project_id = R.project_id and R.year = 2025);
```

(Finds projects staffed by at most one employee in 2025.)

8.3 Subqueries in the FROM Clause

A subquery can stand in for a table, letting you filter on an **aggregate** without `HAVING`.

Example: Average salary of departments where average salary exceeds $60,000:

```
select dept_name, avg_salary
from (select dept_name, avg(salary) as avg_salary
      from employee
      group by dept_name) as dept_avg
where avg_salary > 60000;
```

8.3.1 The `WITH` Clause

`with` defines a **temporary, named result** (like a scratchpad table) visible only to the query that follows.

Example: Find the name of the department that has the maximum budget.

```
with max_budget(value) as
    (select max(budget) from department)
select department.name
from department, max_budget
where department.budget = max_budget.value;
```

Multiple `with` blocks can be chained for multi-step logic:

Example: Find the names of the departments whose total salary expense is greater than the average total salary expense across all departments.

```
with dept_total(dept_name, value) as
    (select dept_name, sum(salary)
     from employee
     group by dept_name),
dept_total_avg(value) as
    (select avg(value) from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value > dept_total_avg.value;
```

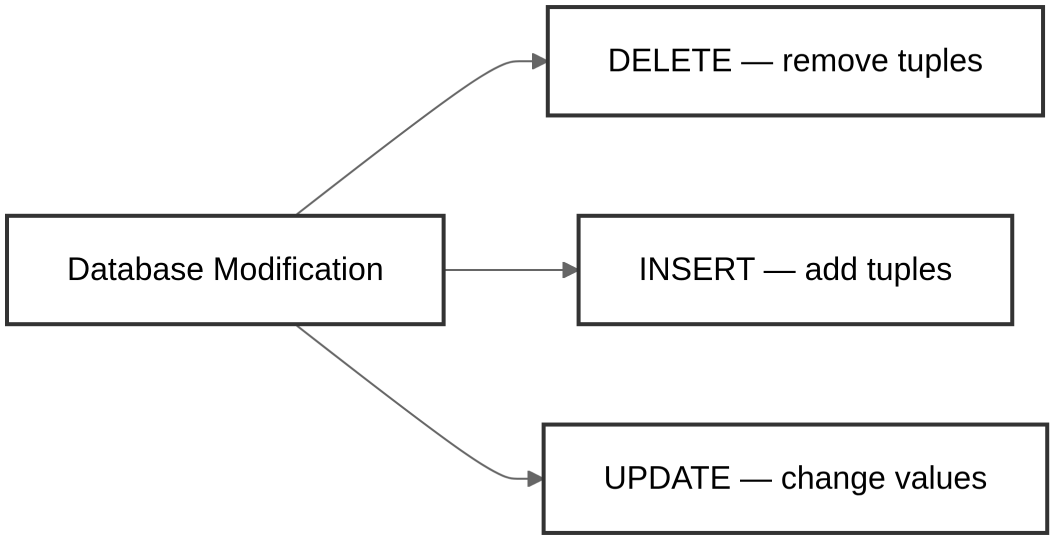
8.4 Subqueries in the SELECT Clause

A **scalar subquery** returns exactly one value, used wherever a single value is expected.

```
select dept_name,
       (select count(*)
        from employee
        where department.dept_name = employee.dept_name) as num_employees
from department;
```

If the subquery returns more than one row, this causes a **runtime error**.

8.5 Modifying the Database



8.5.1 Deletion

```
-- Delete all employees
delete from employee;

-- Delete employees from one department
delete from employee where dept_name = 'Finance';

-- Delete employees in departments located in a specific building
delete from employee
where dept_name in (select dept_name
                    from department
                    where building = 'Tower B');
```

Tricky case: Delete employees earning less than the average salary.

```
delete from employee
where salary < (select avg(salary) from employee);
```

As rows are deleted, the average changes! Solution: **(1)** computing the average and identifying all rows to delete *first*, then **(2)** deleting them — without recomputing mid-way.

8.5.2 Insertion

```
insert into course
values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

-- with tot_creds set to null
insert into student
values ('3003', 'Green', 'Finance', null);

-- insert from another query's results
insert into student
select id, name, dept_name, 0
from employee;
```

> The entire `select-from-where` is evaluated **before** any row is inserted — otherwise `insert into table1 select * from table1` would loop infinitely.

8.5.3 Updates

Conditional updates — give a 3% raise above \$100K, 5% below (order matters!):

```
update employee set salary = salary * 1.03 where salary > 100000;
update employee set salary = salary * 1.05 where salary <= 100000;
```

Cleaner with a **`case`** statement (single pass, no ordering issues):

```
update employee
set salary = case
    when salary <= 100000 then salary * 1.05
    else salary * 1.03
end;
```

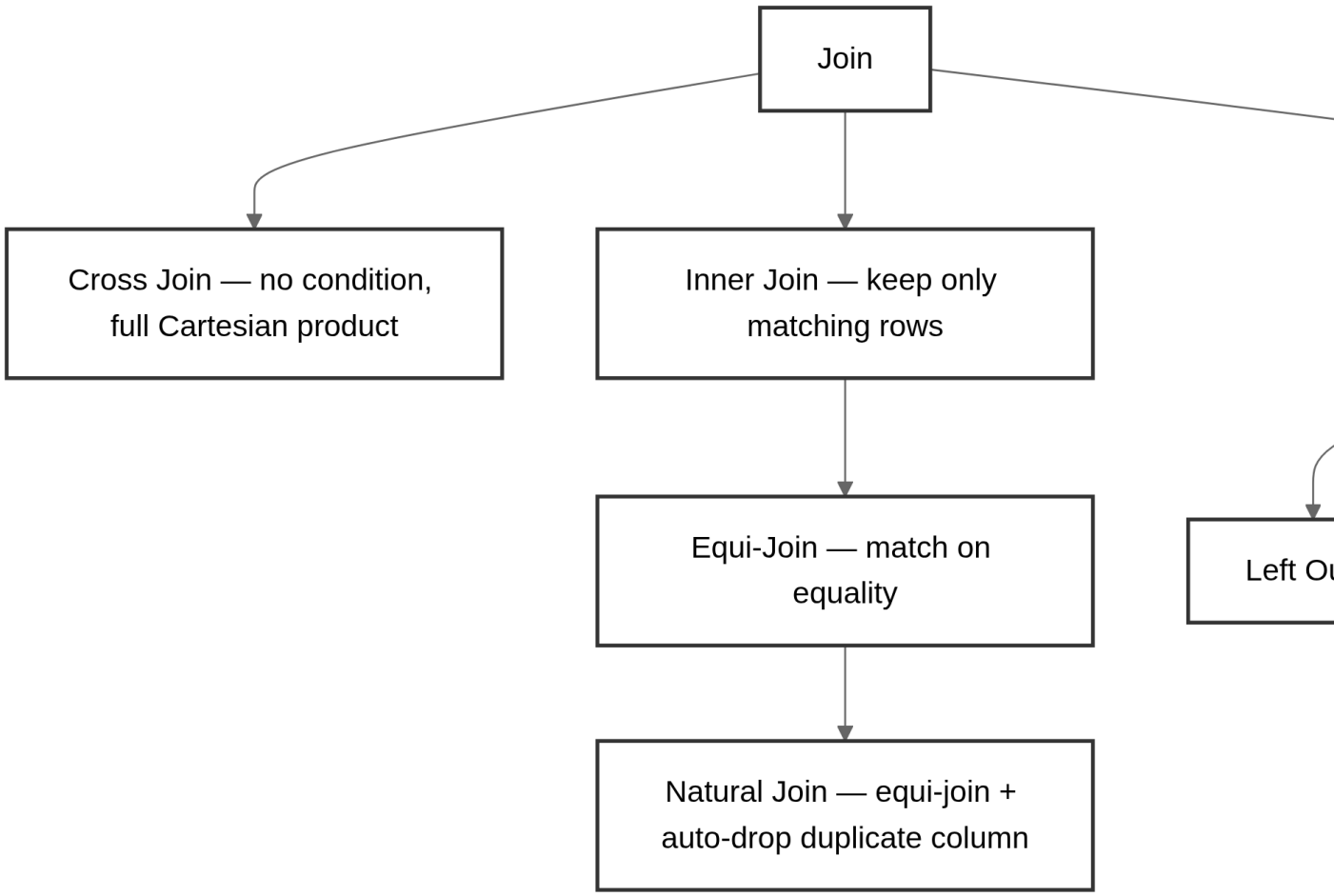
Updates with scalar subqueries — recompute total credits per student:

```
update student S
set tot_creds = (select sum(credits)
                 from takes, course
                 where takes.course_id = course.course_id and
                       S.id = takes.id and
                       takes.grade <> 'F' and
                       takes.grade is not null);
```

(Find the total credits earned by each student (excluding failed or uncollected grades) and update their record in the student table.)

9 Joins

A **join** takes two tables and stitches them into one, based on some matching condition. Think of it as a Cartesian product (every row paired with every row) that's then *filtered* down to only the pairs that actually make sense together.



9.1 Sample Data

To see the difference between join types, imagine two small tables:

movie

movie_id	title
M1	Inception
M2	Arrival
M3	Dune

review

movie_id	rating
M1	9
M2	8
M4	7

Notice: `Dune (M3)` has no review, and the review for `M4` doesn't match any movie. This mismatch is exactly what different join types handle differently.

9.2 Cross Join

Returns the **full Cartesian product** — every row of one table paired with every row of the other, no filtering at all.

```
-- Explicit
select * from movie cross join review;

-- Implicit (comma syntax)
select * from movie, review;
```

With 3 movies and 3 reviews, this produces **9 rows** — almost all of them meaningless pairings.

9.3 Inner Join

Keeps only the rows where the join condition is satisfied on **both sides**.

Join Type	Condition	Result
Equi-Join	Explicit equality condition (`on a.x = b.x`)	Both `x` columns kept
Natural Join	Equality assumed on same-named columns	Duplicate column auto-dropped

```
-- Equi-join
select * from movie inner join review
on movie.movie_id = review.movie_id;

-- Natural join (same result, but movie_id appears once)
select * from movie natural join review;
```

Result (only matching `movie_id`s survive):

movie_id	title	rating
M1	Inception	9
M2	Arrival	8

`M3` (no review) and `M4` (no movie) both **disappear** — this is the information loss outer joins solve.

9.4 Outer Join

Outer joins keep the inner join result **plus** the unmatched rows from one or both sides, filling missing columns with `null`.

9.4.1 Left Outer Join

Keeps **all rows from the left table**, even unmatched ones.

```
select * from movie natural left outer join review;
```

movie_id	title	rating
M1	Inception	9
M2	Arrival	8
M3	Dune	null

9.4.2 Right Outer Join

Keeps **all rows from the right table**, even unmatched ones.

```
select * from movie natural right outer join review;
```

movie_id	title	rating
M1	Inception	9
M2	Arrival	8
M4	null	7

9.4.3 Full Outer Join

Keeps **everything from both sides** — the union of left and right outer joins.

```
select * from movie natural full outer join review;
```

movie_id	title	rating
M1	Inception	9
M2	Arrival	8
M3	Dune	null
M4	null	7

9.4.4 Using Explicit Conditions

Joins can also use `on` (any condition) or `using` (shared column names), instead of `natural`:

```sql
-- Explicit condition
select * from movie left outer join review
on movie.movie_id = review.movie_id;

-- Shared column shorthand
select * from movie full outer join review
using (movie_id);
```

`on` works with *any* condition (even inequality!). `natural` and `using` both rely on matching column names — `using` lets you name exactly which shared column(s) to join on, while `natural` matches on **all** identically-named columns.

9.5 Quick Reference: Join Types

Join	Unmatched left rows kept?	Unmatched right rows kept?
Cross Join	n/a (no matching)	n/a
Inner Join		
Left Outer		
Right Outer		
Full Outer		

---

10 Intermediate SQL

10.1 Views

10.1.1 Why Views?

Not everyone should see the full picture. Suppose someone needs an employee's name and department, but **not** their salary. Instead of trusting every query to remember to hide the salary column, define it once:

```
select id, name, dept_name from employee;
```

A **view** is exactly this — a saved query that behaves like a "virtual table." It's not real stored data; it's a name attached to a query expression that gets substituted in wherever the view is used.

10.1.2 Creating a View

```
create view v as <query expression>;
```

```
-- A view of employees without salary
create view staff_directory as
select id, name, dept_name from employee;

-- Use it like a table
select name from staff_directory where dept_name = 'Marketing';

-- A view with computed columns
create view dept_salary_totals(dept_name, total_salary) as
select dept_name, sum(salary)
from employee
group by dept_name;
```

10.1.3 Views Built on Views

Views can refer other views, not just base tables:

```
create view marketing_2025_projects as
select project.project_id, lead_emp_id, building, room_number
from project, assignment
where project.project_id = assignment.project_id
  and project.dept_name = 'Marketing'
  and assignment.year = 2025;

create view marketing_2025_tower_b as
select project_id, room_number
from marketing_2025_projects
where building = 'Tower B';
```

- `v1` **depends directly on** `v2` if `v2` appears in `v1`'s definition.
- `v1` **depends on** `v2` if there's any chain of dependency leading from `v1` to `v2`.
- A view is **recursive** if it depends on itself.

10.1.4 View Expansion

To understand what a nested view *really* means, SQL conceptually **expands** it: repeatedly replace each view name with its underlying query, until only base tables remain.

```
repeat:
    find a view reference v_i in the expression
    replace v_i with v_i's defining query
until no view references remain
```

This terminates as long as no view is recursive.

10.1.5 Updating Views

Inserting into a view secretly inserts into the underlying table:

```
insert into staff_directory values ('E501', 'Lee', 'Design');
-- Translates to:
-- insert into employee values ('E501', 'Lee', 'Design', null);
```

But this only works cleanly for *simple* views — most databases require:

- `from` clause has exactly **one** base table
- `select` lists plain column names only (no expressions, aggregates, or `distinct`)
- Any column left out of the `select` can default to `null`
- No `group by` or `having`

If a view joins multiple tables, an insert can be **ambiguous** (e.g., which department's row gets the new value if several departments share a building?) — and some inserts that satisfy the view's *output* can violate its `where` clause entirely, making the update meaningless to translate.

10.1.6 Materialized Views

A **materialized view** physically stores the view's result as a real table, instead of recomputing it on every query — faster to read, but it goes **stale** the moment the underlying tables change. Keeping it correct requires actively **maintaining** it (refreshing it) whenever the source data updates.

View Type	Storage	Speed	Freshness
Regular View	None (just a saved query)	Recomputed every time	Always current
Materialized View	Physical table	Fast to read	Needs manual/scheduled refresh

10.2 Transactions

A **transaction** is a unit of work — a group of one or more SQL statements that should succeed or fail *as a whole*.

- **Atomic**: either the *entire* transaction completes, or none of it does (rolled back as if it never happened).
- **Isolated**: a transaction shouldn't see the messy, half-finished state of another transaction running at the same time.
- Transactions begin **implicitly**, and end with either:
  - `commit work` — make all changes permanent
  - `rollback work` — undo everything since the transaction began

10.3 Integrity Constraints

Constraints stop *valid-looking but wrong* changes from silently corrupting the database — e.g., "an employee's salary must be at least $15/hour" or "every customer must have a phone number."

10.3.1 On a Single Relation

Constraint	Purpose
`not null`	Column can never be empty
`primary key`	Uniquely identifies each row; implies `not null` + `unique`
`unique (A1, ..., Am)`	Columns form a **candidate key** — but unlike primary keys, candidate keys *can* be null
`check(P)`	Row must satisfy predicate `P`

```
-- not null
name varchar(20) not null,
budget numeric(12,2) not null

-- check: restrict allowed values
create table project (
    project_id varchar(8),
    status varchar(10),
    primary key (project_id),
    check (status in ('Active', 'On Hold', 'Completed', 'Cancelled'))
);
```

10.4 Referential Integrity

Ensures a value referenced in one table **actually exists** in the table it points to. If `'Design'` appears as a department name in `employee`, there must be a matching row for `'Design'` in `department`.

> **Formal definition:** If `A` is the primary key of relation `S`, then `A` is a **foreign key** of relation `R` if every value of `A` appearing in `R` also appears in `S`.

10.4.1 Cascading Actions

What should happen when a referenced row is deleted or updated?

```
create table project (
    project_id char(5) primary key,
    title varchar(20),
    dept_name varchar(20) references department
);

-- with explicit cascading behavior
create table project (
    ...
    dept_name varchar(20),
    foreign key (dept_name) references department
        on delete cascade
        on update cascade,
    ...
);
```

Action	Effect when referenced row is deleted/updated
`cascade`	Automatically delete/update matching rows too
`no action`	Block the delete/update if references exist (default)
`set null`	Set the foreign key to `null`
`set default`	Set the foreign key to its default value

10.4.2 The Chicken-and-Egg Problem

Self-referencing constraints can deadlock a single insert. Consider:

```
create table employee (
    id char(10),
    name char(40),
    manager_id char(10),
    primary key (id),
    foreign key (manager_id) references employee
);
```

How do you insert the very first employee if their manager must already exist? Three options:

1. Insert the manager *before* the employee who reports to them.
2. Set `manager_id` to `null` initially, then `update` it later (impossible if the column is `not null`).
3. **Defer** constraint checking until the transaction commits (covered in advanced topics).

10.5 SQL Data Types and Schemas

10.5.1 Built-in Date/Time Types

Type	Stores	Example
`date`	Year, month, day	`date '2025-7-27'`
`time`	Hours, minutes, seconds	`time '09:00:30'`
`timestamp`	Date + time together	`timestamp '2025-7-27 09:00:30.75'`
`interval`	A span of time	`interval '1' day`

Subtracting two dates/times gives you an `interval`; adding an `interval` to a date/time shifts it forward or backward.

10.5.2 Indexes

An **index** is a shortcut structure that lets the database jump straight to matching rows instead of scanning the whole table.

```
create table student (
    id varchar(5),
    name varchar(20) not null,
    dept_name varchar(20),
    tot_cred numeric(3,0) default 0,
    primary key (id)
);

create index student_id_index on student(id);
```

```
select * from student where id = '12345';
-- with the index above, this skips a full table scan
```

10.5.3 User-Defined Types (UDT)

Like a `typedef` in C — gives a meaningful name to an existing type.

```
create type dollars as numeric(12,2) final;

create table department (
    dept_name varchar(20),
    building varchar(15),
    budget dollars
);
```

10.5.4 Domains

Similar to UDTs, but domains can carry their own **constraints**.

```
create domain person_name char(20) not null;

create domain degree_level varchar(10)
    constraint degree_level_test
    check (value in ('Bachelors', 'Masters', 'Doctorate'));
```

10.5.5 Large Object Types

For storing big, unstructured content like photos, videos, or files:

Type	Stores
`blob`	**B**inary **L**arge **OB**ject — raw binary data
`clob`	**C**haracter **L**arge **OB**ject — large character data

A query returns a **pointer** to a large object, not the object itself — the actual data is fetched separately.

10.6 Authorization

10.6.1 Forms of Authorization

On data:

Privilege	Allows
`read`	Viewing data only
`insert`	Adding new rows
`update`	Modifying existing rows
`delete`	Removing rows

On schema:

Privilege	Allows
`index`	Creating/dropping indices
`resources`	Creating new relations
`alteration`	Adding/removing columns
`drop`	Deleting relations entirely

10.6.2 Granting Privileges

```
grant <privilege list> on <relation or view> to <user list>;
```

`<user list>` can be a specific user, `public` (everyone), or a **role**.

```
grant select on employee to U1, U2, U3;
grant insert, update on project to U1;
grant all privileges on department to admin_user;
```

> The grantor must already **hold** the privilege themselves (or be the DBA). Granting a privilege on a *view* does **not** automatically grant privileges on the tables underneath it.

10.6.3 Revoking Privileges

```
revoke <privilege list> on <relation or view> from <user list>;
```

```
revoke select on department from U1, U2, U3;
```

- `revoke all from ...` removes every privilege the user holds.
- Revoking from `public` removes the privilege from everyone **except** users who were granted it explicitly elsewhere.
- If the same privilege was granted by **two different** grantors, revoking from one grantor may leave the privilege intact (granted by the other).
- Revoking a privilege also revokes any privileges that **depended** on it.

10.6.4 Roles

Roles bundle privileges so you don't have to grant the same set repeatedly.

```
create role instructor;
grant select on takes to instructor;
grant instructor to amit;       -- amit inherits the privileges of 'instructor'

-- Roles can inherit from other roles
create role teaching_assistant;
grant teaching_assistant to instructor;   -- instructor now also gets TA's privileges

-- Chains of roles
create role dean;
grant instructor to dean;
grant dean to satoshi;          -- satoshi inherits dean → instructor → TA
```

10.6.5 Authorization on Views

```
create view geo_employees as
    (select * from employee where dept_name = 'Geology');

grant select on geo_employees to geo_staff;
```

If `geo_staff` queries `geo_employees`, it works even if `geo_staff` has **no** direct permission on the underlying `employee` table — the view's permissions are what matter, not the creator's original access (though if the view's *creator* lacked permission on `employee`, the view itself couldn't have been created correctly in the first place).

10.6.6 Other Authorization Features

```
-- 'references' privilege needed to create a foreign key pointing at someone else's table
grant references (dept_name) on department to mariano;

-- Transferable privileges
grant select on department to amit with grant option;

-- Revoking with dependency handling
revoke select on department from amit, satoshi cascade;   -- also revokes anything amit/satos
revoke select on department from amit, satoshi restrict;  -- blocks the revoke if dependents
```

The `references` privilege exists because a foreign key constraint can restrict what the *referenced* table's owner is allowed to do (e.g., blocking a delete due to `on delete no action`) — so creating that link requires explicit permission.

11 Advanced SQL

11.1 Why Go Beyond Plain SQL?

SQL is **declarative** — great for describing *what* you want, clumsy for *how* to compute it step by step. Sometimes you need loops, conditionals, and reusable logic. SQL:1999 added **functions, procedures, and control-flow constructs** to bridge that gap, turning SQL into something closer to a general-purpose language.

11.2 SQL Functions

A **function** returns a single value, and can be written in SQL itself — or in an external language (useful for specialized types like images or geometry, e.g. "do these two polygons overlap?").

```
create function emp_count(dname varchar(20))
returns integer
begin
    declare e_count integer;
    select count(*) into e_count
    from employee
    where employee.dept_name = dname;
    return e_count;
end
```

Use it just like any expression:

```
select dept_name, budget
from department
where emp_count(dept_name) > 12;
```

Keyword	Meaning
`begin ... end`	A **compound statement** — holds multiple SQL statements together
`returns`	Declares the *type* of the result (e.g., `integer`)
`return`	Specifies the *actual value* sent back when the function finishes

> Think of SQL functions as **parameterized views** — a view that generalizes by accepting arguments.

11.2.1 Table Functions

Added in SQL:2003 — instead of returning one value, a table function returns an **entire relation**.

```
create function employees_in(dname varchar(20))
returns table (
    id varchar(5),
    name varchar(20),
    dept_name varchar(20),
    salary numeric(8,2)
)
return table
    (select id, name, dept_name, salary
     from employee
     where employee.dept_name = employees_in.dname);
```

```
select * from table(employees_in('Marketing'));
```

11.2.2 Procedures

Where a function returns a value via `return`, a **procedure** passes results back through `out` parameters.

```
create procedure emp_count_proc(
    in dname varchar(20),
    out e_count integer)
begin
    select count(*) into e_count
```

```
    from employee
    where employee.dept_name = emp_count_proc.dname;
end
```

```
declare e_count integer;
call emp_count_proc('Design', e_count);
```

SQL:1999 allows **overloading** — multiple functions/procedures can share a name as long as their parameter count or types differ.

11.3 Procedural Language Constructs

Most databases implement their own dialect of these — syntax varies across systems.

11.3.1 Loops

```
-- while: check condition before each iteration
while boolean_expression do
    sequence_of_statements;
end while;

-- repeat: check condition after each iteration (always runs once)
repeat
    sequence_of_statements;
until boolean_expression
end repeat;

-- for: iterate directly over query results
declare total integer default 0;
for r as
    select budget from department
do
    set total = total + r.budget;
end for;
```

11.3.2 Conditionals

```
-- if / elseif / else
if boolean_expression then
    sequence_of_statements;
elseif boolean_expression then
    sequence_of_statements;
else
    sequence_of_statements;
end if;
```

```
-- simple case: match a variable against fixed values
case dept_name
    when 'Sales' then ...
    when 'Design' then ...
    else ...
end case;

-- searched case: match arbitrary expressions
case
    when salary > 100000 then ...
    when salary > 50000 then ...
    else ...
end case;
```

11.3.3 Exception Handling

```
declare seat_unavailable condition;
declare exit handler for seat_unavailable
begin
    ...
    signal seat_unavailable;
    ...
end
```

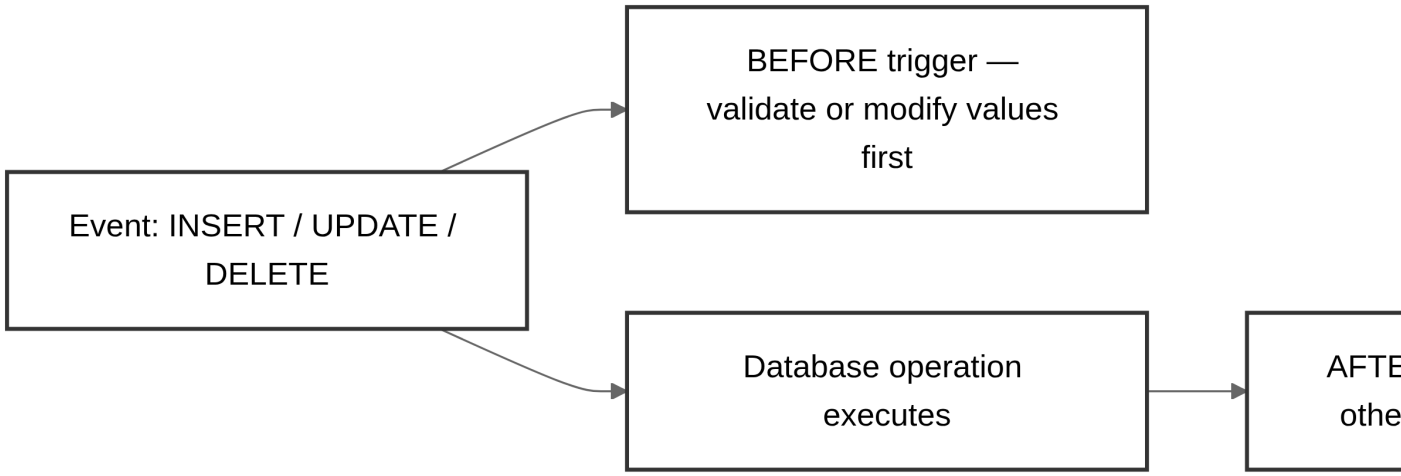
The `exit` handler terminates the enclosing `begin...end` block when the condition fires. Other handler behaviors are also possible (e.g., continue instead of exit).

11.4 Triggers

A **trigger** is code that fires automatically in response to an `insert`, `update`, or `delete` on a table — no one has to remember to call it.

Defining a trigger means specifying three things:

1. **Event** — insert / update / delete
2. **Timing** — `BEFORE` or `AFTER` the event
3. **Action** — what to actually do



11.4.1 BEFORE vs. AFTER

Timing	Typical use
BEFORE	Clean up or validate incoming values before they're written (e.g., normalize blanks to `null`); raise errors before a delete proceeds
AFTER	Update *other* tables to stay in sync, maintain audit trails, enforce cross-table integrity, trigger alerts/notifications

11.4.2 Row-Level vs. Statement-Level

Type	Behavior
Row-level	Fires once **per affected row**. A 10% raise across 100 employees fires the trigger 100 times.
Statement-level	Fires **once per statement**, regardless of how many rows were touched — uses `referencing old table` / `referencing new table` to access all affected rows at once via transition tables. More efficient for bulk updates.

11.4.3 Example: BEFORE trigger (data cleanup)

Convert blank grades to `null` before they're written:

```
create trigger setnull_trigger before update of takes
referencing new row as nrow
for each row
when (nrow.grade = ' ')
begin atomic
    set nrow.grade = null;
end;
```

11.4.4 Example: AFTER trigger (cross-table maintenance)

Update a student's earned credits whenever a failing/blank grade turns into a pass:

```
create trigger credits_earned after update of grade on (takes)
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
     and (orow.grade = 'F' or orow.grade is null)
begin atomic
    update student
    set tot_cred = tot_cred +
        (select credits from course where course.course_id = nrow.course_id)
    where student.id = nrow.id;
end;
```

- `referencing old row as` — access pre-update values (deletes & updates)
- `referencing new row as` — access post-update values (inserts & updates)
- Triggers can be scoped to specific columns: `after update of grade on takes`

11.5 Using Triggers Wisely

11.5.1 Good Uses

- Logging changes to a history/audit table
- Recording metadata an application can't see (who ran it, when, from where)
- Simple, self-contained validation
- Keeping denormalized or derived values in sync

11.5.2 Danger Signs

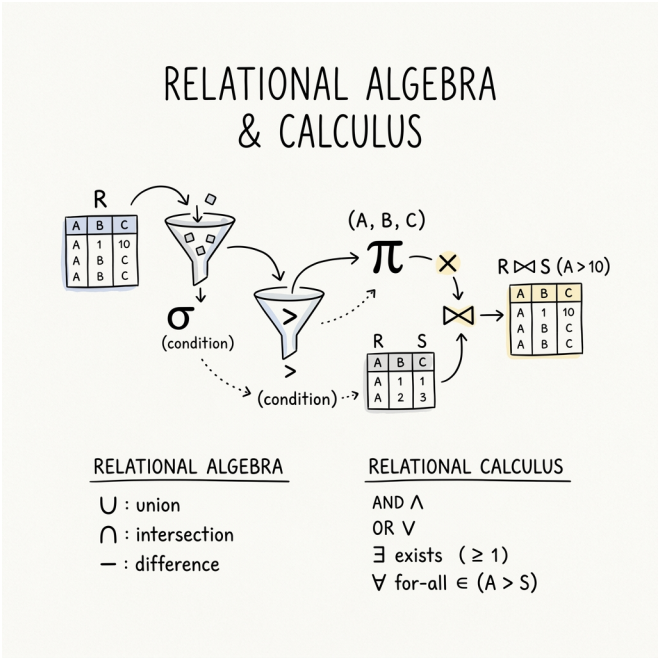
Risk	Why it hurts
Too many triggers	Hard to trace *why* something changed
Complex trigger logic	Business logic buried where developers won't look
Cross-server triggers	Network calls hidden inside what looks like a simple `update`
Triggers calling triggers	Cascading side effects, hard to predict
Recursive triggers enabled	Can spiral — off by default for a reason
Views/procedures/functions nested inside triggers	Performance black box
Iteration inside triggers	Row-by-row processing at scale = slow

Rule of thumb: triggers are best for short, simple, self-contained operations — not as a substitute for your application's business logic.

Part IV

Week-04

12 Relational Algebra and Calculus



13 Introduction to Query Languages

Query languages in DBMS are broadly classified into two categories:

1. **Procedural Query Languages**: The user specifies *what* data is needed and *how* to get it. Relational Algebra is a procedural language.
2. **Non-Procedural (Declarative) Query Languages**: The user specifies *what* data is needed without specifying *how* to get it. Relational Calculus is a non-procedural language.

---

14 Relational Algebra (RA)

Relational Algebra is a procedural query language that takes one or two relations as input and produces a new relation as a result. The fundamental operations include:

- σ - **Select**: Filters rows based on a given condition.
- π - **Project**: Selects specific columns from a relation.
- $\neg$ - **Negation (Not)**
- $\cup$ - **Union**: Combines rows of two relations.
- $\cap$ - **Intersection**: Finds common rows between two relations.
- $\times$ - **Cartesian Product**: Combines each row of the first relation with each row of the second.
- $-$ - **Set Difference (Except)**: Finds rows present in the first relation but not in the second.
- $\bowtie$ - **Natural Join**: Combines two relations based on matching values in common attributes.

14.1 Examples

1. Find all the names of students whose age is greater than 25, or who are enrolled in Maths

Schema: - `Students(RollNo, Name, Age, Subject)`

First, we filter the `Students` table using the Selection (σ) operation for the condition `Age > 25 OR Subject = 'Maths'`. Then, we use the Projection (π) operation to only retrieve the `Name` attribute.

$$\pi_{Name}(\sigma_{Age>25 \vee Subject='Maths'}(Students))$$

2. Find the name and sports of the student whose age is less than 25 and awards is greater than 3

Schema: - `Students(RollNo, Name, Age)` - `Activity(RollNo, Sports, Awards)`

Here we need data from two different domains (the `Students` and `Activity` tables). We use a Natural Join ($\bowtie$) to combine them on their common attribute (`RollNo`), filter the result, and project the specific columns.

$$\pi_{Name,Sports}(\sigma_{Age<25 \wedge Awards>3}(Students \bowtie Activity))$$

14.2 Division Operation

The **Division ($r \div s$)** operation is used in queries that involve the word "every" or "all". For example, "Find students who have taken *all* courses offered in the Biology department."

Suppose we have two relations: $r(A, B, C)$ and $s(B, C)$. The result of $r \div s$ will be a relation with the attribute A, containing values of A that match with *all* the tuples present in s.

- Relations *r, s*:

A	B	C	D	E
α	a	α	a	1
α	a	γ	a	1
α	a	γ	b	1
β	a	γ	a	1
β	a	γ	b	3
γ	a	γ	a	1
γ	a	γ	b	1
γ	a	β	b	1

r

D	E
a	1
b	1

s

Figure 14.1: Division Operation Example

In the mathematical example from the notes, if we perform $r \div s$:

A	B	C
α	a	γ
γ	a	γ

The output would be the values in column `A` (α and γ) that are associated with all tuples of the divisor.

---

15 Tuple Relational Calculus (TRC)

Tuple Relational Calculus is a **non-procedural** query language. Instead of writing operations to fetch data, we write mathematical predicates that define the resulting tuples.

A basic TRC query is of the form:

$$\{t|P(t)\}$$

Where: - t = The resulting tuple(s) - $P(t)$ = The predicate (a condition that evaluates to true or false for tuple t)

15.1 TRC Examples

1. Find the name of the students whose age is 21

$$\{t|\exists s \in students(t.name = s.name \wedge s.age = 21)\}$$

Intuition: We are looking for a tuple t such that there exists ($\exists$) a tuple s in the `students` relation where the name in our result matches the name in the student record, and the student's age is 21.

2. Find the name of the employees who work in the 'Manufacturing' department

Schema: - `employee(id, name, salary)` - `department(id, d_id, name, building)`

$$\{M|\exists E \in employee \exists D \in department(E.id = D.id \wedge D.name =' Manufacturing' \wedge M.name = E.name)\}$$

Intuition: We want a tuple M such that there exists an employee E and a department D where their IDs match, the department name is 'Manufacturing', and our result tuple M takes the name of that employee E.

---

16 Domain Relational Calculus (DRC)

Domain Relational Calculus is another non-procedural query language, equivalent in expressive power to Tuple Relational Calculus. However, instead of using whole tuples as variables, DRC uses variables that take values from domains (individual attributes/columns).

A basic DRC query is of the form:

$$\{< x_1, x_2, ..., x_n > | P(x_1, x_2, ..., x_n)\}$$

Where: - $< x_1, x_2, ..., x_n >$ represents domain variables (e.g., individual columns like name, age, etc.) - P represents a predicate formula similar to TRC.

16.1 DRC Examples

1. Find the name of the students whose age is 21

Schema: - `student(name, age, marks)`

$$\{< a > | \exists b(< a, b, c > \in students \wedge b = 21)\}$$

Intuition: We want to output a single domain variable a (which represents the name), such that there exists some marks variable c where the tuple $< a, b, c >$ exists in the `students` relation, and the age variable $b = 21$.

2. Find the name of the employees who work in the 'Manufacturing' department

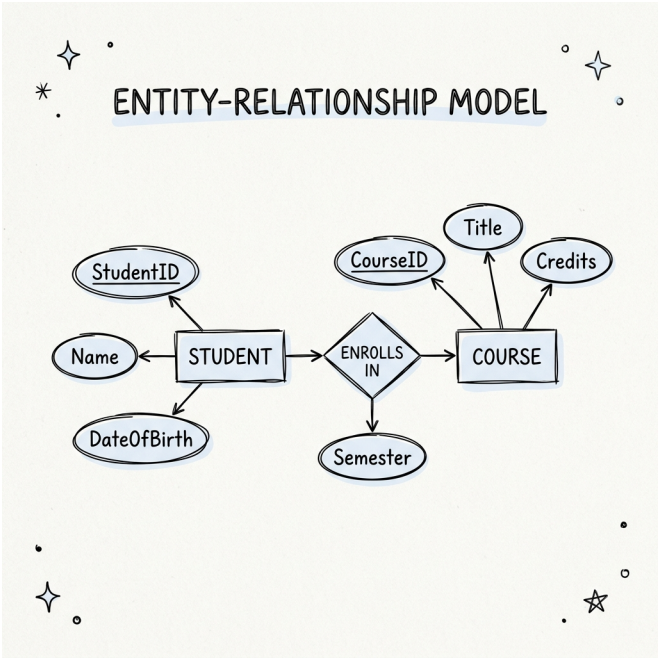
Schema: - `employee(id, name, salary)` - `department(id, d_id, name, building)`

$$\{< b > | \exists a, c(< a, b, c > \in employee) \wedge \exists x, y, z(< a, x, y, z > \in department \wedge y =' Manufacturing')\}$$

(Note: The notes had a slight variable mismatch. I corrected it here to ensure variable a maps to `id`*, b maps to* `name`*, c maps to* `salary`*, and in the department relation, the first variable a corresponds to* `id`*, ensuring the join condition.)*

Intuition: We want to output variable b (employee name). To do this, we assert that the tuple $< a, b, c >$ must exist in `employee`. Additionally, we assert that the tuple $< a, x, y, z >$ must exist in `department` where the variable y (department name) is exactly 'Manufacturing'. The shared variable a (the ID) effectively joins the two relations.

17 Entity-Relationship Model



18 Introduction to Entity Sets

The Entity-Relationship (ER) model is a high-level conceptual data model that helps in designing the schema of a database. It allows us to describe the data involved in a real-world enterprise in terms of objects and their relationships.

- **Entity**: An entity is an object that exists and is distinguishable from other objects. For example, a specific student named "Alice" is an entity.
- **Entity Set**: An entity set is a collection of entities of the same type that share the same properties or attributes. For example, the set of all students in a university forms an entity set.

18.1 Strong Entity Set

- A strong entity set is an entity set that contains sufficient attributes to uniquely identify all its entities.
- A strong entity set always has a **Primary Key**.

18.2 Weak Entity Set

- A weak entity set is an entity set that does not contain sufficient attributes to uniquely identify its entities on its own.
- A primary key does not exist for a weak entity set.
- Instead, it contains a partial key called a **Discriminator**. The discriminator is represented in an ER diagram by underlining the attribute with a **dashed line**.

18.2.1 Characteristics of Weak Entity Sets

- A weak entity set cannot exist independently since it doesn't have a primary key. It depends on a strong entity set.
- It features in the model in relationship with a strong entity set. This dependency is called an **identifying relationship**.
- The primary key of a weak entity set is formed by combining its discriminator with the primary key of the strong entity set it depends on.

- **Primary key of weak entity set = Discriminator + Primary key of Strong entity set**.
- A weak entity set must have **total participation** in its identifying relationship.

---

19 Attributes

An attribute is a property or characteristic associated with an entity set. For example, the `Student` entity set might have attributes like `Name`, `Age`, and `Roll Number`.

19.0.1 Types of Attributes

- **Simple attribute**: An attribute that cannot be divided into smaller sub-parts (e.g., `Age`).
- **Composite attribute**: An attribute that can be divided into smaller sub-parts. For example, a `Name` attribute can be divided into `First Name`, `Middle Name`, and `Last Name`.
- **Single-valued attribute**: An attribute that holds a single value for a specific entity (e.g., `Roll Number`).
- **Multivalued attribute**: An attribute that can hold multiple values for a specific entity. For example, a student might have multiple phone numbers (`{phone_numbers}`).
- **Derived attribute**: An attribute whose value is calculated (derived) from the value of another attribute. For example, `Age()` can be derived from the `Date of Birth` attribute.

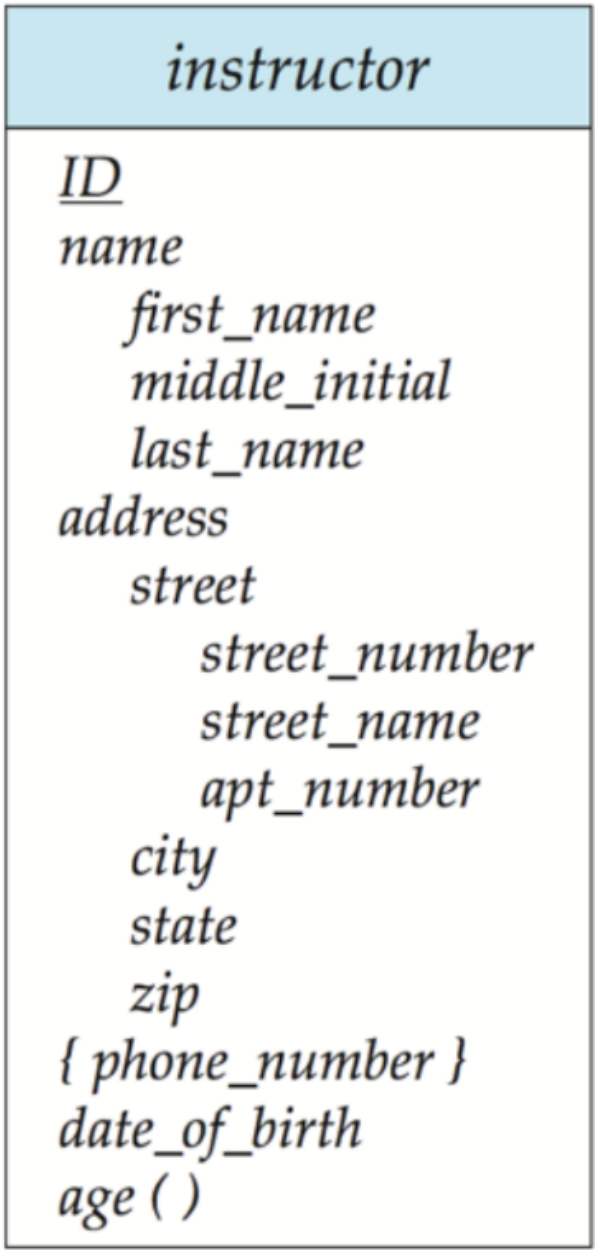


Figure 19.1: Types of Attributes

20 Entity-Relationship (ER) Diagrams

An ER diagram is a visual representation of the ER model. The standard notation involves:

- **Rectangles**: Represent entity sets.
- **Ellipses**: Attributes are listed inside ellipses and connected to the entity set.
- **Solid Underline**: Indicates a primary key attribute.
- **Diamonds**: Represent relationship sets.
- **Double Lines**: Indicate total participation of an entity in a relationship.
- **Double Rectangles**: Represent weak entity sets.
- **Double Diamonds**: Represent identifying relationship sets for weak entity sets.

---

21 Mapping Cardinalities (Relationship Types)

Mapping cardinalities express the number of entities to which another entity can be associated via a relationship set.

21.1 One-to-One (1:1) Relationship

- Each entity in A is associated with at most one entity in B.
- Each entity in B is associated with at most one entity in A.
- *Example*: Each instructor has at most one student, and each student has at most one instructor.

Figure 21.1: One to One

21.2 One-to-Many (1:N) Relationship

- Each entity in A can be associated with one or many entities in B.
- Each entity in B is associated with at most one entity in A.
- *Example*: An instructor can advise multiple students, but each student has only one instructor.

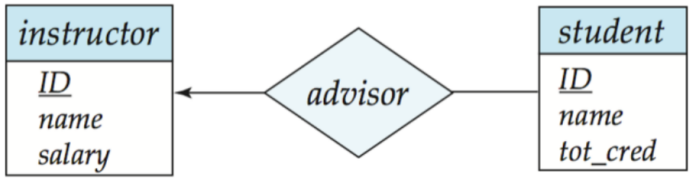


Figure 21.2: One to Many

21.3 Many-to-One (N:1) Relationship

- Each entity in A is associated with at most one entity in B.
- Each entity in B can be associated with one or many entities in A.
- *Example*: Many students can be advised by the same instructor, but each student has at most one instructor.

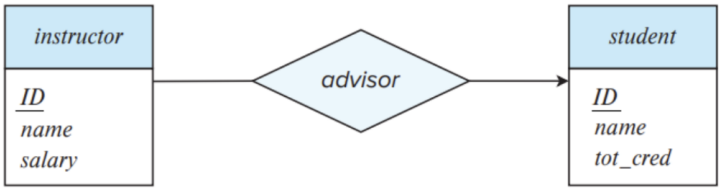


Figure 21.3: Many to One

21.4 Many-to-Many (M:N) Relationship

- Each entity in A can be associated with one or many entities in B.
- Each entity in B can be associated with one or many entities in A.
- *Example*: An instructor can advise multiple students, and a student can have multiple instructors.

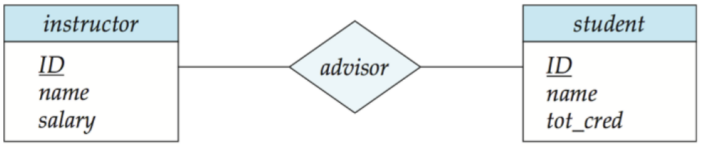


Figure 21.4: Many to Many

---

22 Total and Partial Participation

Participation constraints dictate whether the existence of an entity depends on its being related to another entity via the relationship type.

- **Total Participation**: Every entity in the entity set must participate in at least one relationship in the relationship set. Indicated by a double line.
- **Partial Participation**: Some entities may not participate in any relationship in the relationship set. Indicated by a single line.

Example: If every student *must* have an instructor, the `Student` entity set has total participation. If an instructor *may or may not* have a student, the `Instructor` entity set has partial participation.

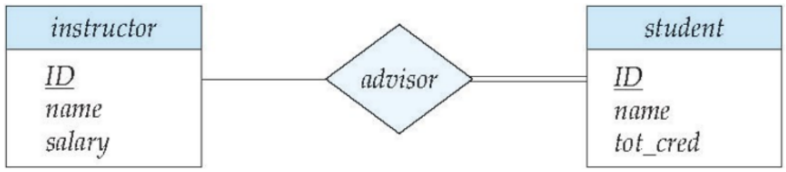


Figure 22.1: Total and Partial Participation

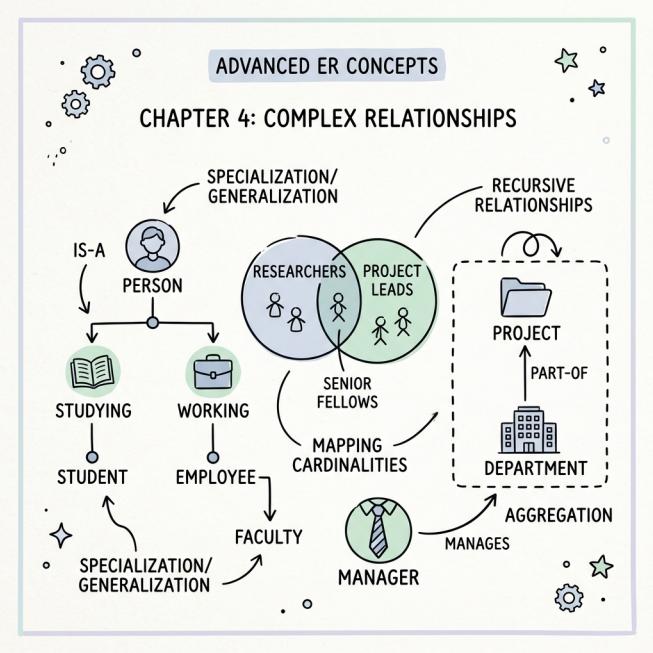
---

23 Expressing Weak Entity Sets in ER Diagrams

As discussed earlier, a weak entity set does not have a primary key and depends on a strong entity set. In an ER diagram, we denote the weak entity set with a double rectangle, its identifying relationship with a double diamond, and its discriminator attribute with a dashed underline.

Figure 23.1: Weak Entity Set Notation

24 Advanced ER Concepts



As databases grow in complexity, the standard Entity-Relationship model may not be sufficient to capture intricate real-world constraints and relationships. To address this, extended ER features are used.

25 Ternary Relationships

A **Ternary Relationship** is a relationship that exists among three entity sets simultaneously. Instead of linking two entity sets (binary), a single diamond connects three rectangles.

Example: Consider a scenario where `Instructors` assign `Projects` to `Students`. This cannot be accurately captured by breaking it into binary relationships because a specific student is assigned a specific project by a specific instructor. Thus, the relationship involves all three entities at once.

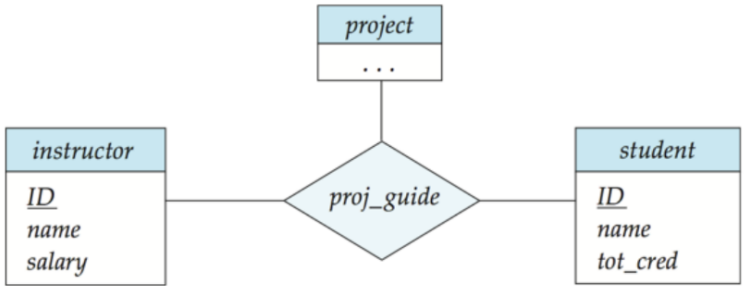


Figure 25.1: Ternary Relationship

26 Aggregation

Aggregation is an abstraction through which relationships are treated as higher-level entities. We use aggregation when we need to establish a relationship *with* another relationship.

Why use Aggregation over Ternary Relationships? Sometimes, a ternary relationship doesn't clearly convey the natural hierarchy of the data. If a relationship inherently acts like an entity that participates in another relationship, aggregation is a better design choice.

Example: Suppose an `Instructor` and a `Student` have an `Advises` relationship. Now, suppose a `Project` is assigned to that specific `(Instructor, Student)` pair. Instead of making a ternary relationship, we draw a box around the `Advises` relationship (aggregating it into a higher-level entity) and connect it to the `Project` entity via an `Assigned` relationship.

Figure: Example of Aggregation

- **Teacher**{*teacher_num*, *teacher_name* }
- **Student**{*student_num*, *student_name* }
- **Project**{*project_num*, *project_name* }
- **Examiner**{*examiner_num*, *examiner_name* }
- **proj_guide**{*teacher_num*, *student_num*, *project_num*, *since* }
- **evaluate**{*teacher_num*, *student_num*, *project_num*, *examiner_num*, *date_of_evaluation* }

Figure 26.1: Aggregation Example

---

27 Extended ER Features: Specialization and Generalization

Specialization is the process of defining a set of subclasses from an entity set (top-down approach). Generalization is the reverse process of combining subclasses into a higher-level superclass (bottom-up approach).

When dealing with class hierarchies, we must define constraints on how entities can participate in the subclasses. These constraints are categorized by **disjointness** and **completeness**.

27.1 Disjointness Constraints

- **Disjoint**: An entity can belong to *at most one* lower-level entity set (subclass).
- **Overlapping**: An entity can belong to *more than one* lower-level entity set simultaneously.

27.2 Completeness Constraints

- **Complete (Total)**: Every entity in the higher-level entity set (superclass) *must* belong to at least one lower-level entity set.
- **Partial**: Some entities in the higher-level entity set *might not* belong to any lower-level entity set.

Let's look at the combinations of these constraints:

27.2.1 1. Overlapping and Partial

- **Overlapping**: An entity can belong to multiple subclasses.
- **Partial**: An entity does not have to belong to any subclass.

Example: A `Person` can be a `Student`, a `Faculty` member, or **both** (Overlapping). Also, there may be persons (like staff or guests) who are **neither** students nor faculty (Partial).

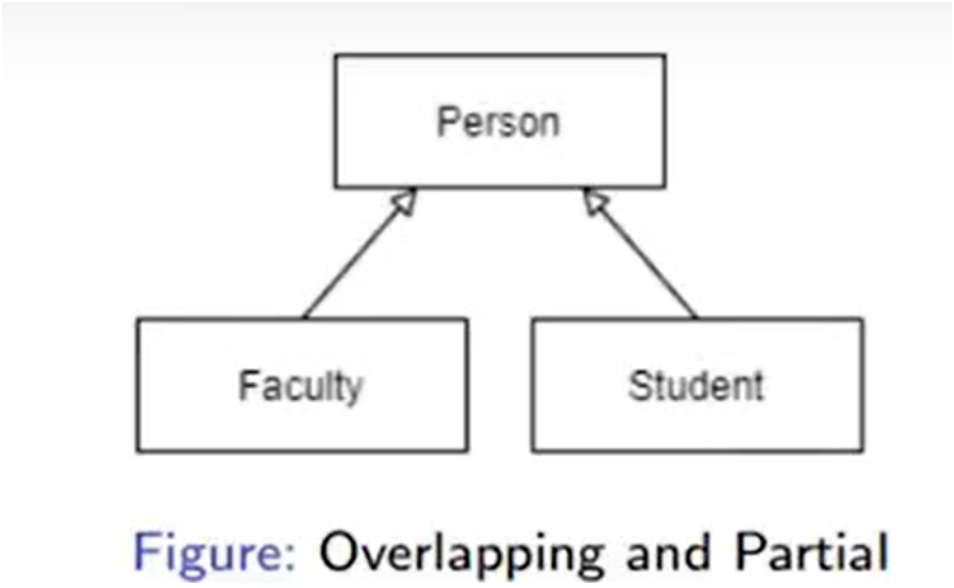


Figure 27.1: Overlapping and Partial

27.2.2 2. Disjoint and Partial

- **Disjoint**: An entity can belong to at most one subclass.
- **Partial**: An entity does not have to belong to any subclass.

Example: A `Student` can be either an `UG_Student` (Undergraduate) or a `PG_Student` (Postgraduate), but they **cannot be both** at the same time (Disjoint). Furthermore, there might be some students (e.g., diploma or visiting students) who are **neither** UG nor PG students (Partial).

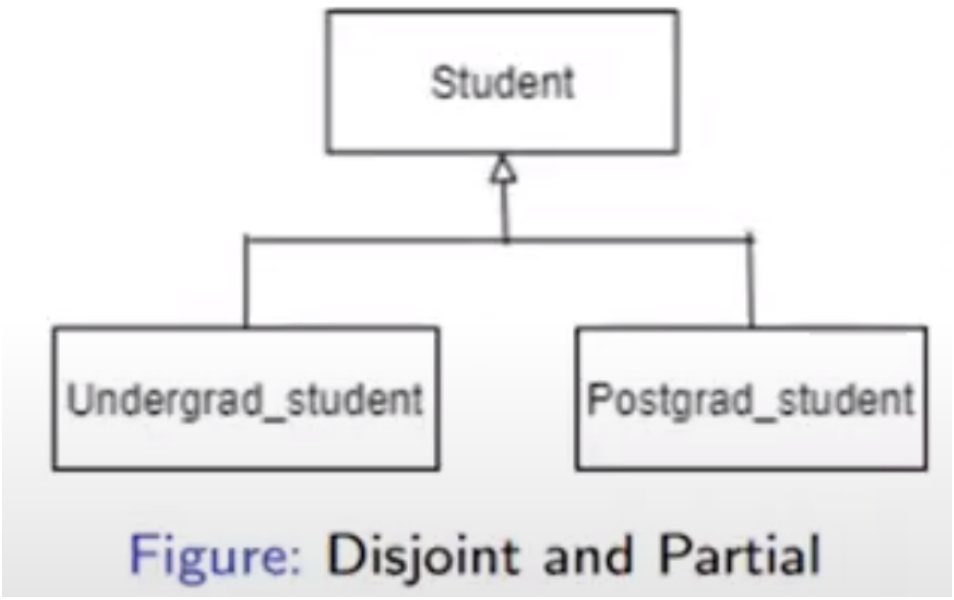


Figure 27.2: Disjoint and Partial

27.2.3 3. Disjoint and Complete

- **Disjoint**: An entity can belong to at most one subclass.
- **Complete**: Every entity must belong to a subclass.

Example: Every physical `Part` in a factory **must** be classified as either a `Purchased_Part` or a `Manufactured_Part` (Complete). A part **cannot** be both purchased and manufactured simultaneously (Disjoint).

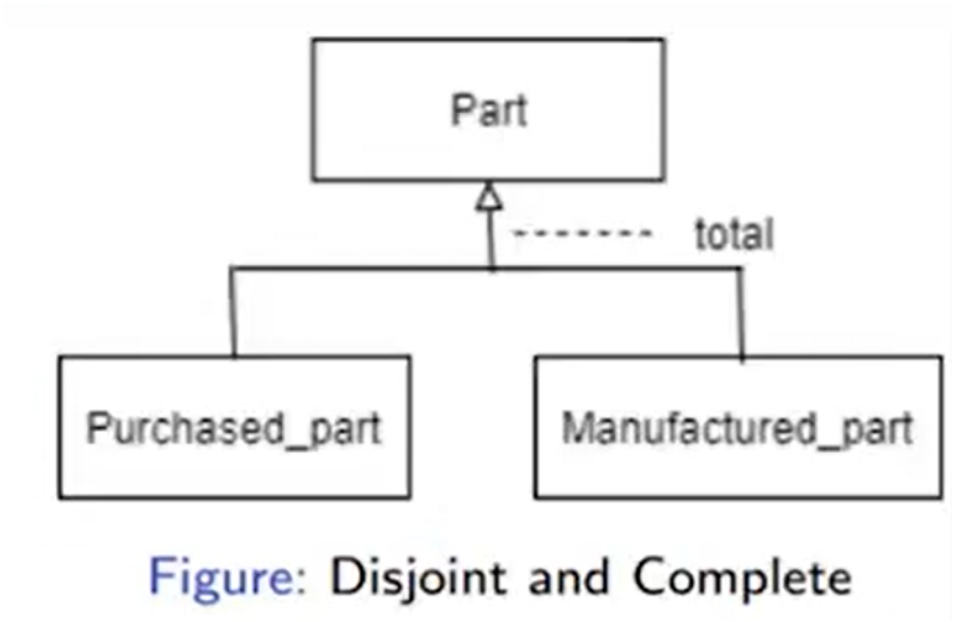


Figure 27.3: Disjoint and Complete

(Note: The fourth combination, Overlapping and Complete, implies an entity must belong to at least one subclass and can belong to multiple. It is less common but conceptually possible.)

---

28 Summary

The concepts covered in the Entity-Relationship model can be summarized below. ER diagrams provide a standard language to communicate database schemas between developers, designers, and stakeholders.

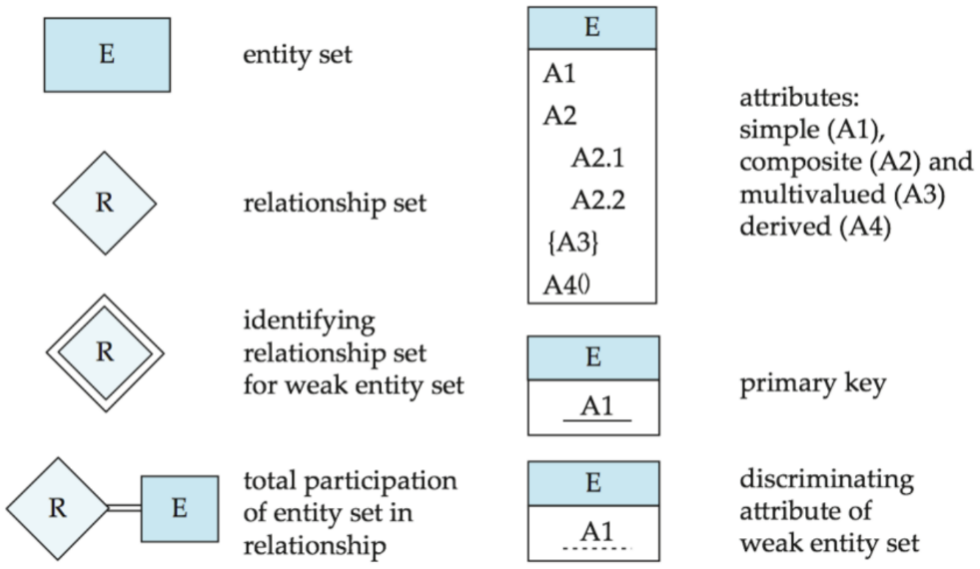


Figure 28.1: ER Diagram Summary

Part V

Week-05

29 Relational Design: Redundancy, Anomalies & Functional Dependencies

29.1 Redundancy and Anomalies

A relational schema should reflect real-world structure, avoid storing the same fact twice, and stay easy to update correctly. When a schema fails at this, two related problems show up: **redundancy** and **anomalies**.

29.1.1 A Bad Schema

`instructor_with_department(ID, name, salary, dept_name, building, budget)`

- `ID`, `name`, `salary` — belong purely to the instructor, no repetition
- `dept_name`, `building`, `budget` — belong to the *department*

Since a department has many instructors, `building` and `budget` get copied once per instructor in that department. That repetition is **redundancy** — the same data item stored in multiple places.

Here's what that looks like with actual rows:

ID	name	salary	dept_name	building	budget
101	Srinivasan	65000	Comp. Sci.	Taylor Hall	100000
102	Wu	90000	Finance	Painter Hall	120000
103	Gold	87000	Comp. Sci.	Taylor Hall	100000
104	Katz	75000	Comp. Sci.	Taylor Hall	100000

Notice `Taylor Hall` and `100000` repeat for every Comp. Sci. instructor — that's the redundancy in action.

29.1.2 Anomalies

Redundancy causes **anomalies** — inconsistencies triggered by insert, delete, or update operations. Using the same table:

Anomaly	What Goes Wrong	On This Table
Insertion	Can't add a record without unrelated data	To add a brand-new "Physics" department with no instructors yet, we can't — there's no row to put it in until a physics instructor is hired
Deletion	Deleting one record erases unrelated info	If Wu (row 102) resigns and we delete that row, we lose the fact that Finance's building is Painter Hall and its budget is 120000 — nowhere else stores that
Update	One fact change needs many row updates	If Comp. Sci.'s budget rises to 110000, we must update rows 101, 103, *and* 104 — miss one, and the table now claims two different budgets for the same department

29.1.3 Why This Happens — and the Fix

`dept_name` uniquely decides `building` and `budget` (a department can't have two budgets). This dependency is exactly what forces `building`/`budget` to repeat across instructor rows:

Dependency → Redundancy → Anomaly

The fix is to **decompose** the table so each fact is stored exactly once — e.g., split `instructor_with_department` into `instructor(ID, name, salary, dept_name)` and `department(dept_name, building, budget)`. (Covered in detail under Lossless Join Decomposition.)

Removing redundancy this way is what makes a schema easier to maintain: updates touch a single row, deletions don't silently destroy unrelated facts, and inserts don't need unrelated data just to go through. This is the core motivation behind everything else in normalization theory.

29.2 Functional Dependencies

29.2.1 Definition

A **functional dependency (FD)** says: knowing the value of one set of attributes is enough to pin down another.

For relation schema R, with $\alpha \subseteq R$ and $\beta \subseteq R$:

$$\alpha \to \beta$$

holds if, for any legal instance of R, two tuples that agree on α must also agree on β:

$$t_1[\alpha] = t_2[\alpha] \implies t_1[\beta] = t_2[\beta]$$

Example: in `instructor_with_department`, we expect:

```
dept_name → building
dept_name → budget
```

since a department has exactly one building and one budget. We would *not* expect `dept_name → salary` — a department tells you nothing about one specific instructor's salary.

> **Note:** An FD only "holds" if it's true for *every* legal instance of the schema, not just today's data. If the current rows happen to satisfy `name → ID` by coincidence (no two instructors currently share a name), that doesn't mean the FD genuinely holds.

29.2.2 Keys, Redefined Using FDs

- K is a **superkey** for R if and only if $K \to R$ (K determines every attribute in R)
- K is a **candidate key** for R if and only if $K \to R$, **and** no proper subset $\alpha \subset K$ also satisfies $\alpha \to R$ (K is minimal)

29.2.3 Trivial FDs

An FD $\alpha \to \beta$ is **trivial** if $\beta \subseteq \alpha$ — it's automatically true for any relation and tells us nothing new.

```
ID, name → ID     -- trivial (ID is already on the left)
name → name       -- trivial
```

30 Armstrong's Axioms & Closure of Attribute Sets

30.1 Armstrong's Axioms

Given a set of FDs F, these three rules let us mechanically derive new FDs that logically follow from it:

Axiom	Rule	Intuition
Reflexivity	if $\beta \subseteq \alpha$, then $\alpha \to \beta$	A set of attributes always determines its own subsets
Augmentation	if $\alpha \to \beta$, then $\gamma\alpha \to \gamma\beta$	Adding the same extra attributes to both sides keeps an FD valid
Transitivity	if $\alpha \to \beta$ and $\beta \to \gamma$, then $\alpha \to \gamma$	If A determines B, and B determines C, then A determines C

Applying these repeatedly to F gives every FD **logically implied** by F — this full set is the closure F^+, and $F \subseteq F^+$.

30.1.1 Worked Example

For $R = (A, B, C, G, H, I)$ with $F = \{A \to B,\ A \to C,\ CG \to H,\ CG \to I,\ B \to H\}$:

$$A \to H \quad \text{(by transitivity, since } A \to B \text{ and } B \to H\text{)}$$

This wasn't stated directly in F, but follows logically — and is therefore part of F^+.

> **Note:** Computing all of F^+ this way (repeatedly applying axioms until nothing new appears) works, but is expensive. **Attribute closure**, covered next, is the practical shortcut.

30.1.2 Derived Rules

These follow from the three basic axioms — not new rules, just useful shortcuts:

Rule	Statement
Union	if $\alpha \to \beta$ and $\alpha \to \gamma$, then $\alpha \to \beta\gamma$
Decomposition	if $\alpha \to \beta\gamma$, then $\alpha \to \beta$ and $\alpha \to \gamma$
Pseudotransitivity	if $\alpha \to \beta$ and $\gamma\beta \to \delta$, then $\alpha\gamma \to \delta$

30.2 Closure of Attribute Sets

Instead of computing the entire F^+, we usually just want to know: what does *this* set of attributes determine? That's the **closure of an attribute set**, written α^+ — all attributes functionally determined by α under F.

30.2.1 How to Compute It

1. Start with `result =  `.
2. Scan through every FD $\beta \to \gamma$ in F.
3. If β is already contained in `result`, add γ to `result`.
4. Repeat steps 2–3 until no new attributes get added.

30.2.2 Worked Example

For $R = (A, B, C, G, H, I)$ with $F = \{A \to B,\ A \to C,\ CG \to H,\ CG \to I,\ B \to H\}$, compute $(AG)^+$:

Step	Result	Why
1	`AG`	starting point
2	`ABCG`	$A \to B$ and $A \to C$ apply
3	`ABCGH`	$CG \subseteq$ result, so $CG \to H$ applies
4	`ABCGHI`	$CG \to I$ applies

$(AG)^+ = ABCGHI$ — every attribute in R.

30.2.3 Superkeys and Candidate Keys, via Closure

- K is a **superkey** of R if $K^+ \supseteq R$ (contains every attribute)
- K is a **candidate key** of R if $K^+ \supseteq R$ **and** no proper subset of K also satisfies this (minimal)

Applying this to AG: $(AG)^+ = R$, so AG is a superkey. Checking subsets — $(A)^+ = ABC$ and $(G)^+ = G$ — neither reaches all of R. So AG is minimal, making it a **candidate key**.

30.2.4 Prime and Non-Prime Attributes

- **Prime attribute**: belongs to *some* candidate key
- **Non-prime attribute**: belongs to *no* candidate key

Here's a data instance for $R = (A, B, C, G, H, I)$ consistent with $F = \{A \to B,\ A \to C,\ CG \to H,\ CG \to I,\ B \to H\}$, compute $(AG)^+$:

A	B	C	G	H	I
a1	b1	c1	g1	h1	i1
a1	b1	c1	g2	h1	i2
a2	b2	c2	g1	h2	i3
a2	b2	c2	g2	h2	i4

Only the pair (A, G) is unique across every row — no other single column or combination pins down a row on its own (`A` alone repeats across rows 1–2 and 3–4; `G` alone repeats across rows 1&3, 2&4). That matches what we found algebraically: AG is the candidate key.

So: `A` and `G` are **prime** attributes; `B`, `C`, `H`, `I` are **non-prime** — each of them is simply *determined by* the key, not part of it.

30.2.5 Formula: Maximum Number of Superkeys

Given a relation with N total attributes, the number of possible superkeys depends directly on the number of candidate keys and the size of those keys.

30.2.6 Case 1: Only 1 Candidate Key (Single Attribute)

If the relation has only one candidate key, and that key consists of a single attribute (e.g., Key $= \{A_1\}$):

$$\text{Total Superkeys} = 2^{N-1}$$

30.2.7 Case 2: Only 1 Candidate Key (Composite Key)

If the relation has only one candidate key, and that key is a composite key formed by k attributes:

$$\text{Total Superkeys} = 2^{N-k}$$

30.2.8 Case 3: Every Single Attribute is a Candidate Key

If the relation has N candidate keys (meaning every single attribute in the relation is its own independent candidate key):

$$\text{Total Superkeys} = 2^N - 1$$

30.2.9 Case 4: Two Distinct Candidate Keys

When a relation contains two distinct candidate keys (which can be composed of one or many attributes), we must use the **Principle of Inclusion-Exclusion** to avoid double-counting the superkeys that contain both keys.

30.2.10 Generic Set Formula

Max Superkeys = (Superkeys with A_1)+(Superkeys with A_2)−(Superkeys with both A_1 and A_2)

30.2.11 Mathematical Representation

Let m be the number of attributes forming the first candidate key, and n be the number of attributes forming the second candidate key. Assuming the two candidate keys share no overlapping attributes:

$$\text{Max Superkeys} = 2^{N-m} + 2^{N-n} - 2^{N-(m+n)}$$

> **Note:** If the two candidate keys share overlapping attributes, the exponent in the subtraction term changes from $(m+n)$ to the number of attributes in their absolute union: $|\text{Key}_1 \cup \text{Key}_2|$.

Example: $R = (A, B, C, G, H, I)$ has $n = 6$ attributes; candidate key AG has $k = 2$. Number of superkeys: $2^{(6-2)} = 2^4 = 16$.

30.2.12 Why Attribute Closure Matters

Use Case	How	In Plain English
Test if α is a superkey	Check $\alpha^+ \supseteq R$	Compute what α determines — if it covers every column in the table, α is enough to identify a row
Test if $\alpha \to \beta$ holds	Check $\beta \subseteq \alpha^+$	Compute what α determines, then see if β shows up in that list
Compute all of F^+	For every $\gamma \subseteq R$, output $\gamma \to S$ for each $S \subseteq \gamma^+$	Try every possible set of columns, work out what each one determines, and write down all those dependencies

31 Extraneous Attributes, Equivalence & Canonical Cover

We already have **attribute closure** (α^+) as our go-to tool for testing superkeys and FDs. Now we use it to ask a sharper question: does every attribute in every FD actually earn its place? This leads to **extraneous attributes** and, from there, the **canonical cover** — the leanest version of an FD set.

31.1 Extraneous Attributes

> **In plain English:** an extraneous attribute is dead weight — remove it, and the FD set means exactly the same thing. It was never pulling any weight to begin with.

An attribute can be dead weight on either side of an FD:

- **Extraneous on the left (α):** attribute $A \in \alpha$ is extraneous in $\alpha \to \beta$ if the *rest* of the left side, without A, still determines β on its own.
- **Extraneous on the right (β):** attribute $A \in \beta$ is extraneous in $\alpha \to \beta$ if the *other* FDs in the set can already derive A anyway — this one FD isn't needed to get it.

31.1.1 How to Test

Left side: Drop A from α. Compute $(\alpha - A)^+$. If β is inside that result, A was extraneous.

Right side: Drop A from β (just for this one FD). Recompute α^+ using this adjusted set. If A still shows up in the result, it means the other FDs already covered it — so A was extraneous.

31.1.2 Example — Extraneous on the Left

$F = \{A \to C,\ AB \to C\}$

Is B extraneous in $AB \to C$? Drop it and check A^+. Since $A^+ = AC$ already contains C, yes — A alone was always enough. B was dead weight.

31.1.3 Example — Extraneous on the Right

$F = \{A \to C,\ AB \to CD\}$

Is C extraneous in $AB \to CD$? Drop it, leaving $\{A \to C,\ AB \to D\}$. Compute AB^+ under this reduced set: it comes out to $ABCD$ — which still contains C. So C was always derivable anyway, and was extraneous in the original FD.

31.2 Equivalence & Canonical Cover

31.2.1 Equivalence of FD Sets

> **In plain English:** two FD sets are equivalent if they're just two different ways of writing down the same set of rules.

F and G are **equivalent** if $F^+ = G^+$. This needs both directions to hold:

- F **covers** G: everything in G can be derived from F
- G **covers** F: everything in F can be derived from G

Only if both hold are F and G truly equivalent.

31.2.2 Canonical Cover

> **In plain English:** the canonical cover is the FD set with all the fat trimmed off — no whole dependency that another one already implies, and no attribute inside a dependency that isn't actually needed.

F_c is a canonical cover of F if:

- $F_c^+ = F^+$ (says exactly the same thing as F)
- No FD in F_c has an extraneous attribute
- Every left-hand side in F_c appears only once (no two FDs share a left side — those would just merge)

31.2.3 How to Compute It

1. Combine any FDs that share the same left-hand side into one, using the union rule.
2. Look at each remaining FD and test both sides for extraneous attributes; remove any that are found.
3. Repeat steps 1–2 until nothing changes.

> Removing an extraneous attribute can create a *new* opportunity to merge left-hand sides that didn't match before — so keep looping until the set is stable.

31.2.4 Worked Example

$R = (A, B, C)$, $F = \{A \to BC,\ B \to C,\ A \to B,\ AB \to C\}$

1. **Merge:** $A \to BC$ and $A \to B$ share a left side → combine into $A \to BC$. Set is now $\{A \to BC,\ B \to C,\ AB \to C\}$.
2. **Left-side check:** is A extraneous in $AB \to C$? Drop it — $B \to C$ is already in the set, so yes. Set is now $\{A \to BC,\ B \to C\}$.
3. **Right-side check:** is C extraneous in $A \to BC$? Drop it and check — $A \to B$ and $B \to C$ together still give $A \to C$ by transitivity. So yes, C was extraneous.

Canonical cover: $\{A \to B,\ B \to C\}$ — the smallest set that says exactly what the original four FDs said.

32 Lossless Join Decomposition & Dependency Preservation

32.1 Lossless Join Decomposition

> **In plain English:** if you split a table in two, you should be able to join the pieces back together and get *exactly* the original table back — not extra, made-up rows, and not missing ones.

For R decomposed into R_1, R_2, this must hold for every legal instance:

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$$

Sufficient condition: the decomposition is lossless if at least one of these is true:

$$R_1 \cap R_2 \to R_1 \quad \text{or} \quad R_1 \cap R_2 \to R_2$$

In other words: the attribute(s) shared between the two pieces must be enough, on their own, to act as a key for at least one of the two pieces.

> This condition is sufficient whenever all our constraints are FDs. If other kinds of constraints are in play, it isn't guaranteed to be necessary.

Full checklist for a lossless decomposition:

- $R_1 \cup R_2 = R$ — nothing gets left out
- $R_1 \cap R_2 \neq \phi$ — the two pieces actually share something in common
- $R_1 \cap R_2 \to R_1$ or $R_1 \cap R_2 \to R_2$ — that shared part is a key on at least one side

32.1.1 Worked Example: Supplier-Parts

`Supplier_Parts(S#, Sname, City, P#, Qty)`, with `S# → Sname, City` and `(S#, P#) → Qty`.

Here's a sample instance:

S#	Sname	City	P#	Qty
S1	Smith	London	P1	300
S1	Smith	London	P2	200
S2	Jones	Paris	P1	400
S2	Jones	Paris	P2	300

A bad split: `Supplier(S#, Sname, City, Qty)` and `Parts(P#, Qty)`

The shared attribute here is `Qty` — and `Qty` isn't a key of either piece. Projecting our sample data onto these two tables and joining them back on `Qty` gives:

S#	Sname	City	P#	Qty	
S1	Smith	London	P1	300	original
S1	Smith	London	P2	300	spurious
S2	Jones	Paris	P1	300	spurious
S2	Jones	Paris	P2	300	original
S1	Smith	London	P2	200	original
S2	Jones	Paris	P1	400	original

Both suppliers happened to have a `Qty = 300` row, so the join matches *every* `Qty = 300` row in `Supplier` with *every* `Qty = 300` row in `Parts` — creating two fabricated combinations that never existed in the original data. This decomposition is lossy, and it also fails to preserve `(S#, P#) → Qty`.

A good split: `Supplier(S#, Sname, City)` and `Parts(S#, P#, Qty)`

The shared attribute is `S#`, which *is* a key of `Supplier`. Joining these two projections back together on `S#` reconstructs exactly the original four rows — no more, no less. Lossless, and every dependency is preserved too.

> The lesson: it's not *whether* you split a table, it's *where* you split it. The same table can be decomposed well or badly.

32.2 Dependency Preservation

> **In plain English:** after splitting a table, can you still check all your original rules just by looking at each piece separately — or do you need to join tables back together first to catch a rule violation? If you can check everything locally, the decomposition "preserves dependencies."

Let F_i be the set of FDs from F^+ that only involve attributes inside R_i. A decomposition into $R_1, ..., R_n$ **preserves dependencies** if:

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

If this fails, catching a violation of some original FD needs an expensive join across tables.

32.2.1 A Quick Contrast

$R = (A, B, C)$, $F = \{A \to B,\ B \to C\}$

Decomposition	Dependency Preserving?
$R_1 = (A, B),\ R_2 = (B, C)$	Yes — both `A→B` and `B→C` sit fully inside one piece each
$R_1 = (A, B),\ R_2 = (A, C)$	No — `B→C` can't be checked without joining, since `B` and `C` now live in different tables

Same relation, same FDs — the *choice* of split decides the outcome.

32.2.2 How to Test It

1. Start with `result` set to just the left-hand-side attributes (α) of the FD you're checking.
2. Go through each piece R_i of the decomposition. Take the attributes `result` shares with R_i, compute their closure under the *full* F, then keep only the attributes that also belong to R_i. Add anything new to `result`.
3. Repeat step 2, cycling through every R_i, until `result` stops growing.
4. If the right-hand-side attributes (β) are now inside `result`, the FD is preserved without needing a join.

Run this check on every FD in F to confirm the whole decomposition preserves dependencies.

32.2.3 Worked Example

$R(A, B, C, D)$, $F = \{A \to B,\ B \to C,\ C \to D,\ D \to A\}$

Decomposition: $R_1(A, B),\ R_2(B, C),\ R_3(C, D)$

`A→B`, `B→C`, `C→D` each sit fully inside one piece, so they're automatically preserved. The one worth checking is `D→A`:

Round	Piece Checked	Closure Computed	Result So Far
Start	—	—	`{D}`
1	$R_1 = (A, B)$	`result  R1 = {}` → nothing to add	`{D}`
1	$R_2 = (B, C)$	`result  R2 = {}` → nothing to add	`{D}`
1	$R_3 = (C, D)$	`(D)+ = ABCD`, keep only `C,D`	`{C,D}`
2	$R_2 = (B, C)$	`(C)+ = ABCD`, keep only `B,C`	`{B,C,D}`
2	$R_1 = (A, B)$	`(B)+ = ABCD`, keep only `A,B`	`{A,B,C,D}`

`result` now contains `A` — so `D → A` is preserved. Every FD in F checks out, so this decomposition is dependency preserving.

Part VI

Playground

33 SQL Playground

34 Python-DB Playground